

MANUAL DO UTILIZADOR

# Asterism

## & AI Agents Map

Os seus agentes de IA, modelos locais, servidores MCP e skills num só mapa

Asterism 0.10.1 · AI Agents Map 0.22.0
outubro 2026

# Sobre este manual

Este manual descreve uma aplicação que existe em duas edições:

| Edição | Asterism | AI Agents Map |
|---|---|---|
| **O que é** | Uma app independente GTK 4 / libadwaita para qualquer ambiente de trabalho Linux | Um plugin para o ambiente de trabalho Omarchy, aberto a partir de um botão na barra |
| **Instala-se a partir de** | AUR, AppImage ou código-fonte | O Omarchy Plugin Marketplace |
| **Aparência** | Segue o modo claro/escuro e a cor de destaque do sistema | As cores do tema Omarchy ou a aparência Adwaita do GNOME |
| **Pasta das definições** | `~/.config/asterism/` | `~/.config/omarchy/ai-agents-map/` |

Tudo o resto — Configuração, o mapa, ligar desenhando, os LLMs locais, os modelos, os servidores MCP e as skills — é igual em ambas. O que só se aplica a uma edição está marcado com ASTERISM ou OMARCHY.

## Convenções

| | |
|---|---|
| **Adicionar** | Um botão, separador ou item de menu da janela. |
| Definições › **Aparência** | Um caminho pela interface: abra as definições e depois **Aparência**. |
| `~/.agents/skills` | Um ficheiro, uma pasta ou um comando. `~` é a sua pasta pessoal. |
| Ctrl+, | Um atalho de teclado. |

> **NOTA** As notas dão contexto útil.

> **DICA** As dicas mostram um caminho mais rápido ou um bom hábito.

> **ATENÇÃO** As caixas de atenção assinalam algo difícil de desfazer.

# Índice


1 **Introdução** 1
- 1.1 O que faz 1
- 1.2 Ideias-chave 2
- 1.3 Nada muda sem pré-visualização 2

2 **Instalação** 3
- 2.1 Asterism 3
- 2.2 AI Agents Map no Omarchy 3
- 2.3 Primeiro arranque 3
- 2.4 Remover a app 4

3 **A janela num relance** 5
- 3.1 Configuração 6
- 3.2 Escolher o que o mapa mostra 8
- 3.3 Trabalhar com o mapa 8
- 3.4 Problemas por resolver 9
- 3.5 Atualizações e sugestões 10

4 **Agentes** 11
- 4.1 Detalhes de um agente 11
- 4.2 Iniciar e parar agentes 12
- 4.3 Tráfego de rede 12
- 4.4 Subagentes 13
- 4.5 Configurar… 13
- 4.6 Entrar 13
- 4.7 Atualizar agentes 14
- 4.8 Agentes que a app pode instalar 14
- 4.9 Agentes novos 15

5 **LLMs locais** 16
- 5.1 Instalar um LLM local 16
- 5.2 Iniciar e parar com os agentes 16
- 5.3 Detalhes de um LLM local 17
- 5.4 Memória da GPU e libertar memória da GPU 17
- 5.5 Modelos ao mesmo tempo 17
- 5.6 Usado por outras apps 18

6 **Modelos** 19
- 6.1 Obter do Hugging Face 19
- 6.2 Instalados 20
- 6.3 Tamanho do contexto 20
- 6.4 Neste PC 20
- 6.5 Mover e eliminar modelos 20
- 6.6 Uma pasta dos modelos 20

7 **Ligar desenhando** 22
- 7.1 Agente ↔ LLM local 22
- 7.2 Agente ↔ servidor MCP 22
- 7.3 Agente ↔ pasta de skills 22
- 7.4 Quando uma ligação falha 22

8 **Servidores MCP** 23
- 8.1 De onde vêm os servidores 23
- 8.2 Configurar um servidor 23
- 8.3 Gerir servidores 24

9 **Skills** 25
- 9.1 Encontrar skills 25
- 9.2 Pastas de skills 25

10 **Definições e ajuda** 26
- 10.1 Definições 26
- 10.2 Ajuda 27
- 10.3 Espaço em disco 28

11 **Smart Check** 29
- 11.1 Resumo da configuração do PC 30
- 11.2 Atualizações encontradas 30
- 11.3 Estatísticas de tokens 30

12 **Verificação de saúde e segurança** 32
- 12.1 Ler os cartões 32
- 12.2 Alterar uma definição 33
- 12.3 O que é verificado em cada agente 33
- 12.4 Controlar o risco 34
- 12.5 As palavras explicadas 35

13 **Complementos** 37
- 13.1 Configurar o Asterism 37
- 13.2 A página de complementos 38
- 13.3 Instalar, ativar e desativar, atualizar, remover 39
- 13.4 Níveis de privacidade 40
- 13.5 Tipos de complementos 40
- 13.6 Um agente novo no seu computador 41
- 13.7 Criar um complemento 41

14 **Privacidade e segurança** 42

15 **Resolução de problemas** 43

A **Referência** 44
- A.1 Atalhos de teclado 44
- A.2 Ficheiros e pastas 44
- A.3 Glossário 45

CAPÍTULO 1

# Introdução

O Asterism mostra a configuração de IA do seu computador como cartões e como mapa: que agentes de programação estão instalados, que modelos de linguagem usam — serviços na nuvem e motores locais —, que modelos estes servem e que servidores MCP e skills tem cada agente. E permite-lhe mudar essa configuração desenhando linhas.

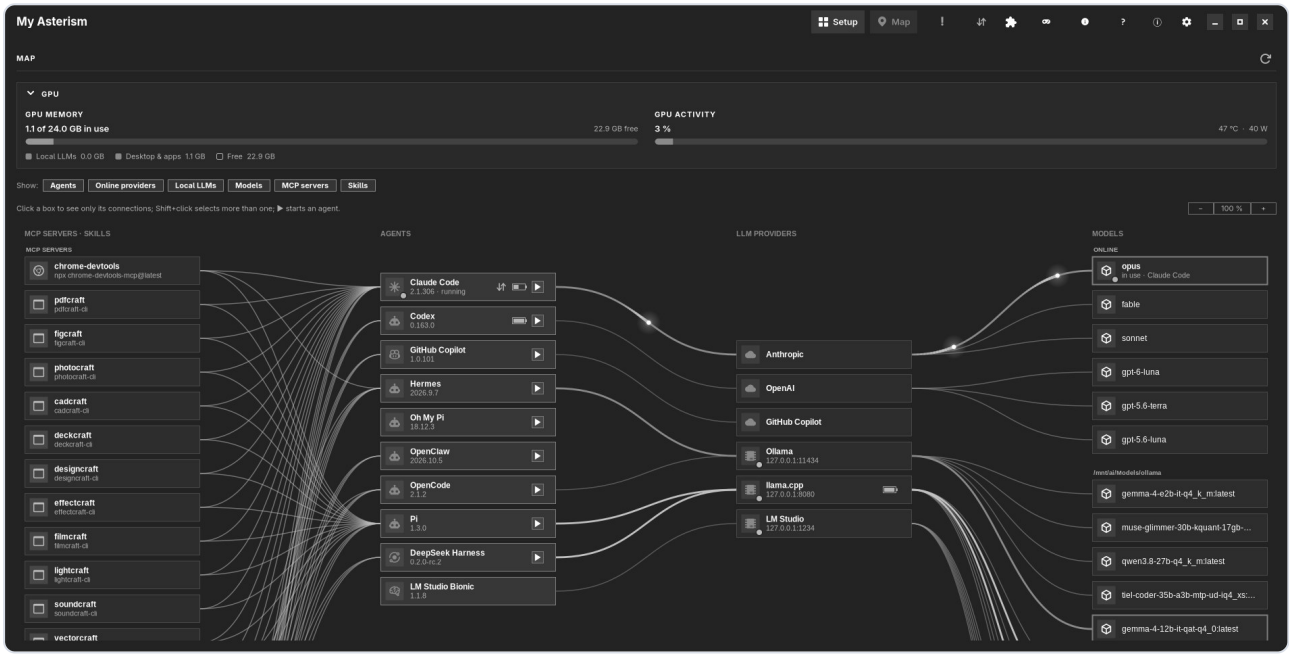



**Figura 1** · A janela principal: o mapa em toda a largura, por cima o cartão GPU, no cabeçalho o ! dos problemas por resolver.

## 1.1 O que faz

- **Agentes de programação** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes e outros: a versão, o plano, os limites e o uso, e o que falta a cada um, com o comando que o resolve.
- **LLMs locais** — llama.cpp, Ollama, LM Studio, vLLM, Jan e KoboldCpp: instalá-los, descarregar modelos, definir o tamanho de contexto que cabe na sua placa gráfica e iniciá-los com os agentes que os usam.
- **Servidores MCP e skills** — encontrados no seu computador ou na web, com servidores recomendados e uma pesquisa no registo MCP oficial.
- **Problemas por resolver** — um ! no cabeçalho quando há algo a corrigir, cada problema com o caminho até onde se corrige, e para que serve a sua configuração.

## 1.2 Ideias-chave

| | |
|---|---|
| **Agente** | Um programa com o qual fala e que escreve código ou faz tarefas, como o Claude Code. |
| **Fornecedor de LLM** | Onde corre o modelo de um agente: um serviço na nuvem (Anthropic, OpenAI, …) ou um motor de LLM local neste computador (llama.cpp, Ollama, …). |
| **Modelo** | O modelo de linguagem propriamente dito. Os modelos locais são ficheiros, quase sempre em formato GGUF. |
| **Servidor MCP** | Um pequeno programa ou serviço web que dá ferramentas extra aos agentes — ler ficheiros, pesquisar na web, controlar um navegador. MCP é o Model Context Protocol. |
| **Skill** | Uma pasta com instruções (um `SKILL.md`) e por vezes scripts, que ensina a um agente uma tarefa concreta. |

## 1.3 Nada muda sem pré-visualização

Cada alteração às definições de um agente é mostrada primeiro, linha a linha, antes de se escrever o que quer que seja. O ficheiro antigo é guardado como cópia de segurança, e a barra que aparece depois da alteração tem **Desfazer**. A app em si nunca corre com permissões de administrador; só a instalação de um agente que vem como pacote do sistema pede a sua palavra-passe, num terminal que vê.

CAPÍTULO 2

# Instalação

## 2.1 Asterism ASTERISM

Escolha uma destas formas:

- **Arch Linux (AUR):** instale o pacote `asterism` com o seu assistente AUR.
- **AppImage:** descarregue `Asterism-<versão>-x86_64.AppImage` da página de versões em `github.com/uBruckhaus/Asterism`, torne-o executável e abra-o. O AppImage usa o Python 3 da sua distribuição com PyGObject, GTK 4 e libadwaita:

| | |
|---|---|
| Fedora | `sudo dnf install python3-gobject gtk4 libadwaita` |
| Ubuntu / Debian | `sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1` |
| Arch | `sudo pacman -S python-gobject gtk4 libadwaita` |

- **A partir do código-fonte:** execute `bin/asterism` num clone do repositório.

A app traz os seus próprios ícones, por isso não é preciso nenhuma fonte de ícones. Para correr agentes dentro da janela, instale também o VTE para GTK 4 ( `vte4` ).

## 2.2 AI Agents Map no Omarchy OMARCHY

Instale o plugin a partir do Omarchy Plugin Marketplace ou num terminal:

```
omarchy plugin add https://github.com/uBruckhaus/omarchy-ai-agents-map.git --enable
```

Aparece um botão com um robô na barra. Tudo o resto de que precisa vem com uma instalação normal do Omarchy. Para correr agentes dentro da janela, instale o VTE: `sudo pacman -S vte4` .

## 2.3 Primeiro arranque

Na primeira abertura, a app pergunta se deve adicionar-se ao lançador de aplicações. Nada é escrito lá a não ser que diga que sim; pode mudar isso mais tarde em Definições › **No lançador**.

A janela é uma janela de aplicação normal: pode ficar em mosaico, flutuar, ser movida para outra área de trabalho e alcançada com `Alt`+`Tab`. ASTERISM Um Asterism novo abre na sua **página de boas-vindas**: os agentes e LLMs locais instalados neste computador são cartões já assinalados, e **Ativar** coloca-os no mapa (ver Configurar o Asterism). OMARCHY O botão da barra abre a janela, trá-la para a frente ou esconde-a se já estiver à frente. Fechar esconde a janela; o programa continua em segundo plano para voltar a abrir de imediato.

> NOTA Se houver um agente a correr dentro da janela, ao fechar pergunta-se primeiro: **Parar e fechar** (que também para o LLM local do agente e liberta a sua memória gráfica) ou **Manter a correr**.

## 2.4 Remover a app

OMARCHY `omarchy plugin remove ubruckhaus.ai-agents-map`. ASTERISM Remova o pacote ou apague o AppImage.

Antes disso, desative Definições › **Iniciar LLMs locais com os seus agentes** para que o vigilante em segundo plano também seja removido, e Definições › **No lançador** para apagar a entrada do lançador. O que alterou com a app fica: definições dos agentes, servidores MCP, skills e LLMs locais instalados. Desinstale primeiro um LLM local (clique direito › **Desinstalar…**) se quiser que desapareça. Para apagar também o estado da própria app:

```
rm -rf ~/.config/omarchy/ai-agents-map ~/.config/asterism \
       ~/.local/state/ai-agents-map ~/.cache/ai-agents-map
```

CAPÍTULO 3

# A janela num relance

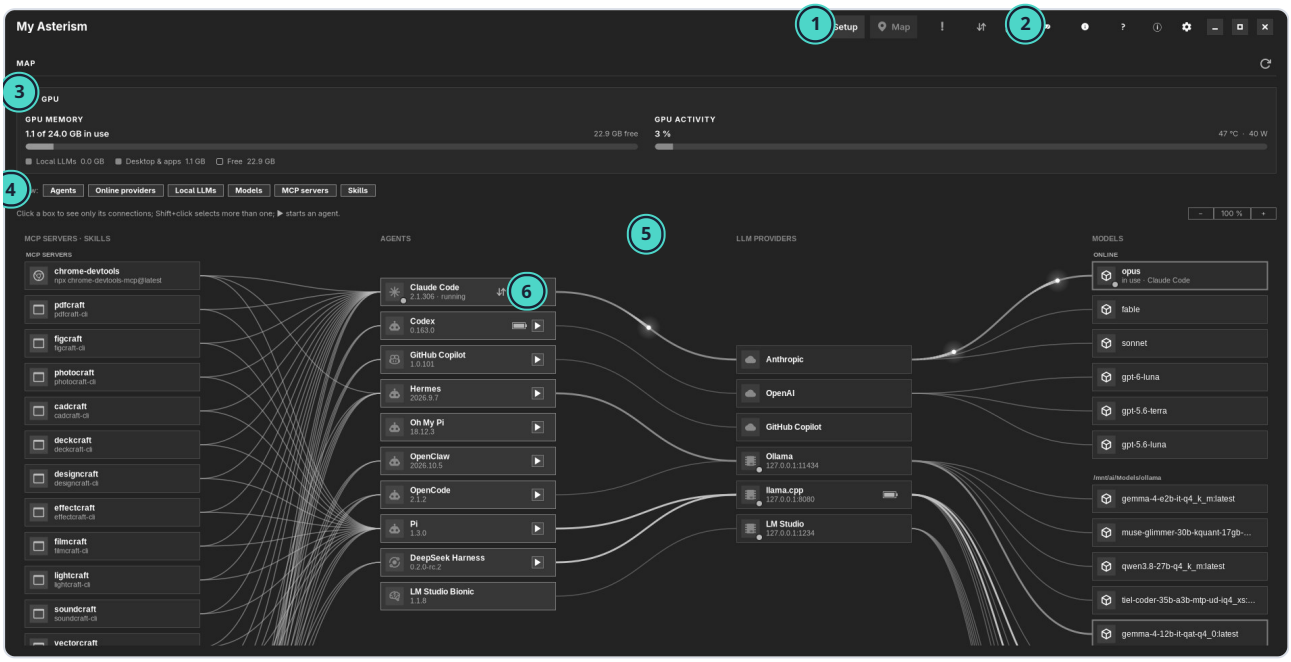



**Figura 2 ·** A janela principal. Os números correspondem à lista abaixo.

1. **Configuração** e **Mapa** — as duas vistas, em todas as páginas; a que está à vista aparece realçada. Configuração tem cada agente, LLM local, servidor MCP, pasta de skills e modelo como um cartão: instalar, atualizar, iniciar sessão, remover (ver abaixo).
2. Os símbolos do cabeçalho: ↻ atualizar (`F5`: voltar a ver, como no arranque), **!** problemas por resolver e atualizações (vermelho: algo a corrigir; a cor de destaque: só atualizações e sugestões; não aparece quando não há nada), ⓘ a legenda (o que significa cada ponto, linha e marca do mapa), ⇅ o tráfego de rede dos agentes em execução (ver Tráfego de rede), as duas estrelinhas do **Smart Check** (só enquanto o Claude Code ou o Codex estiver instalado, ver Smart Check), os complementos ASTERISM, o **i** preenchido para Informações (ver Ajuda), **?** Ajuda (`F1`) e a roda dentada das Definições (`Ctrl`+`,`). Mantenha o botão esquerdo do rato premido sobre um símbolo e arraste-o para o lado para o pôr noutro sítio: uma barra mostra onde fica, e a ordem é guardada.
3. **GPU** — memória da GPU e atividade da GPU: como a placa gráfica está a ser usada agora, num cartão que se recolhe (recolhido, a sua linha mantém os números).
4. Interruptores **Mostrar** — reduzem o mapa às partes em que está a trabalhar.
5. **O mapa** — servidores MCP e skills ← agentes → fornecedores de LLM → modelos, em toda a largura, sob o título **MAPA**.
6. ▶ na caixa de um agente inicia-o num terminal dentro da janela; enquanto corre lá, o botão traz de volta o seu terminal.

## 3.1 Configuração

O mapa mostra apenas o que está configurado: agentes e LLMs locais instalados, os servidores MCP e pastas de skills que os agentes usam, e modelos. Tudo o resto faz-se em **Configuração** (o botão do cabeçalho ao lado de **Mapa**), um cartão por elemento, em secções que se recolhem com um clique na sua barra:

- **Como as coisas chegam ao seu mapa** — um cartão recolhido no topo; abra-o para ver cinco passos numerados: ligar (ou encontrar) o que usa, instalar o que falta, iniciar sessão num agente online, ligar desenhando, e o que aparece sozinho.
- **O que pode fazer com isto** — para que serve a configuração (ajuda no dia a dia, programação, imagens, voz, pesquisa e escrita), um mosaico cada com o que existe e o passo seguinte; recolhido, a sua linha diz quantos estão prontos.
- **Agentes de IA, LLMs locais** — todos os que a app conhece. Um cartão com contorno azul está instalado neste PC; um cinzento não, e tem **Instalar** onde os instalados têm **Desinstalar**. OMARCHY Um clique num cartão instalado abre os seus detalhes. ASTERISM Um clique assinala ou desassinala o cartão (✓) e depois **Aplicar** ativa ou desativa o seu complemento. Desativar tira-o do mapa; nada é apagado.
- **Fornecedores online** ASTERISM — o complemento dos fornecedores online; o seu menu tem **Entrar em …** para cada agente instalado que os usa.
- **Servidores MCP, Skills, Modelos** — um cartão para cada item instalado. Um clique abre os detalhes (num modelo, o seu menu). O cartão **+** no fim de uma secção acrescenta um: **Adicionar servidor MCP**, **Adicionar skills**, **Adicionar modelo**, **Obter complementos**.
- ⋯ ou um clique direito num cartão abre o mesmo menu que a sua caixa no mapa: atualizar, entrar, iniciar ou parar, detalhes, desinstalar ou remover, mover um modelo.
- Um ↑ **versão** amarelo num cartão significa que há uma atualização; o seu menu tem **Atualizar**.
- Um **!** vermelho num cartão significa um problema por resolver ali; passe o rato por cima para ver qual.
- Uma pequena bateria no cartão de um agente online mostra o que resta dos seus limites (uma sessão, uma semana): conta o mais apertado, amarela com um quarto ou menos, vermelha abaixo de um décimo; passe o rato por cima para ver cada limite e quando é reposto. No cartão de um LLM local em execução, a bateria é a memória da GPU ainda livre, com as mesmas cores; passe o rato por cima para ver quanta esse LLM local usa.
- O símbolo de atualizar (F5) na linha do cabeçalho volta a ver o que está instalado, configurado e com sessão iniciada, e repete as verificações do arranque (novas apps, add-ons, atualizações, problemas em aberto).

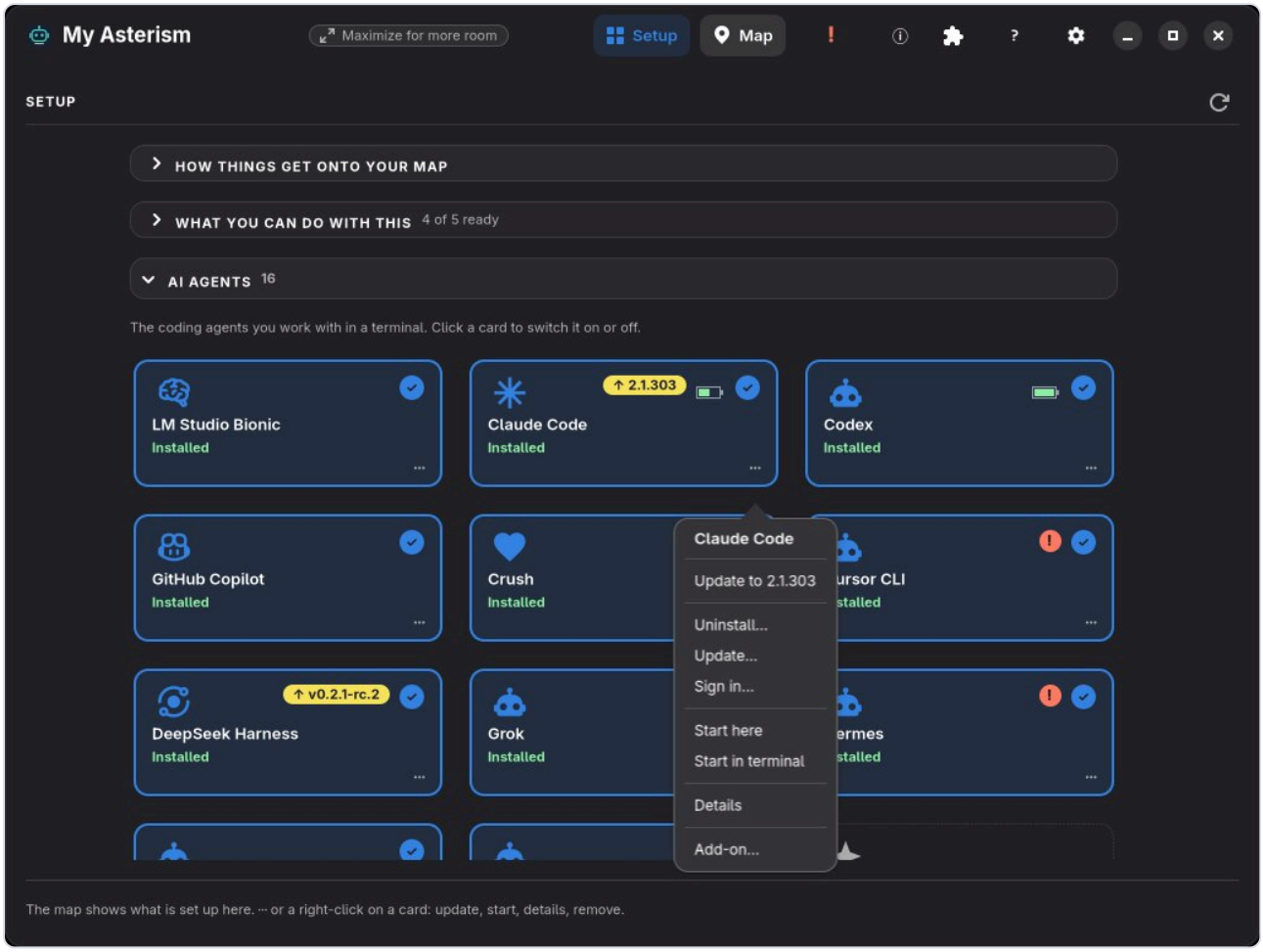



**Figura 3 ·** Configuração: os agentes instalados têm moldura azul, os ligados um ✓, as atualizações um distintivo amarelo e os problemas um vermelho; o menu do Claude Code.

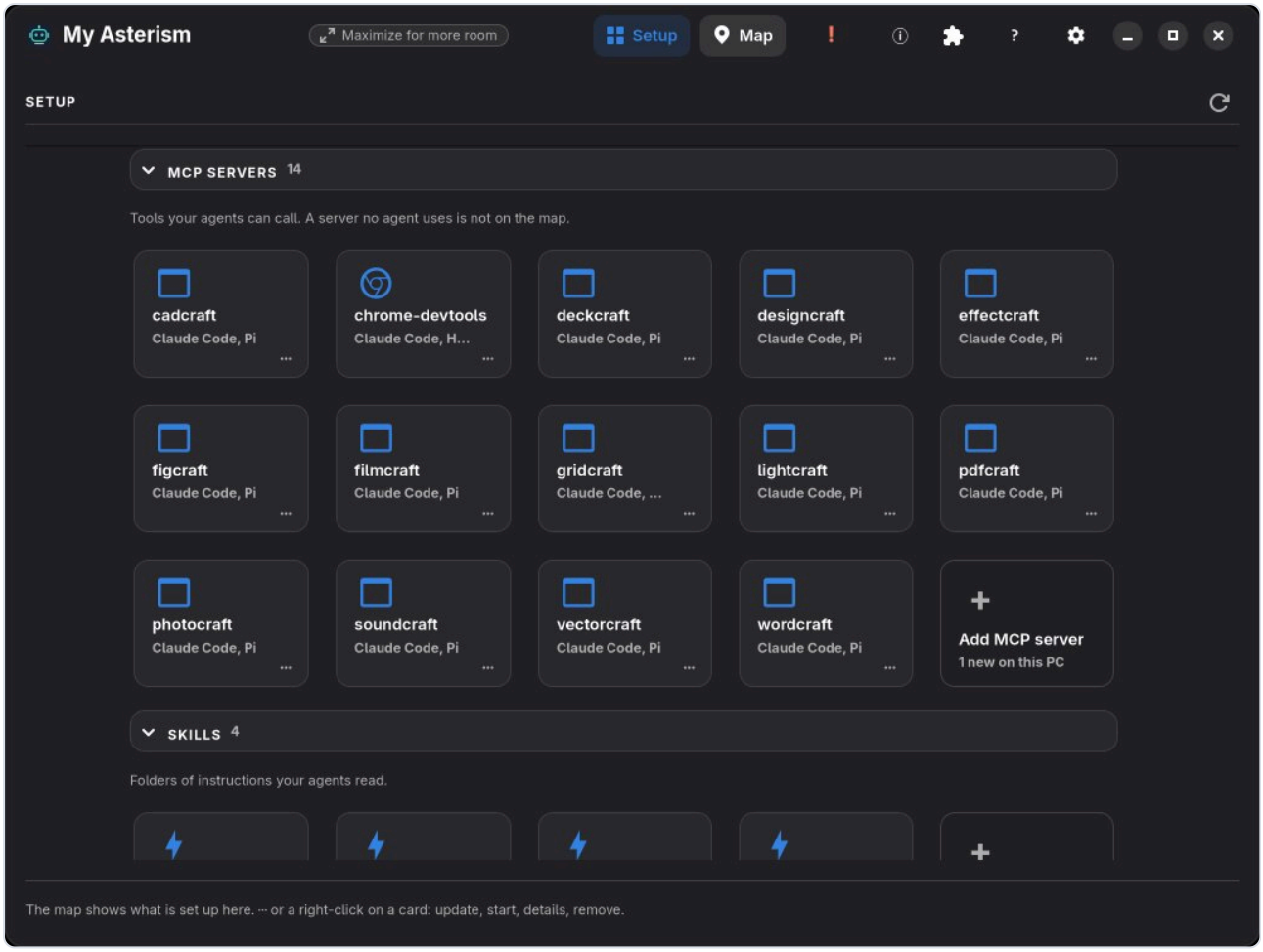



**Figura 4** · Servidores MCP e skills, cada secção com um cartão para adicionar.

Não há nada para esconder: o que não quer no mapa, desativa-o ou remove-o aqui. Quando uma instalação ou um início de sessão corre num terminal dentro da janela, **Voltar** (ou **×** no seu separador) leva-o à página de onde partiu.

## 3.2 Escolher o que o mapa mostra

O mapa mostra tudo o que está configurado. Os interruptores **Mostrar** por baixo do cartão GPU deixam de fora zonas inteiras — por exemplo todas as skills, ou os LLMs locais e os seus modelos — e ficam memorizados.

Numa janela estreita (meio ecrã ou menos) o mapa escolhe-as sozinho, e uma linha junto aos interruptores di-lo: enquanto um agente está em execução, agentes e fornecedores (quem trabalha através de quê); enquanto nenhum está, fornecedores e modelos. Um interruptor clicado ali mantém-se até isso mudar e não fica memorizado; a janela em tamanho completo mantém a sua própria escolha.

## 3.3 Trabalhar com o mapa

- **Passe o rato** por cima de uma caixa para a realçar juntamente com tudo aquilo a que está ligada.
- **Clique** numa caixa para a selecionar (✓); `Shift`+clique acrescenta mais. `Esc` ou **Limpar seleção** limpa-a.

- O **pionés** no canto superior direito de uma caixa mantém as suas ligações realçadas enquanto olha para outro lado.
- **Clique direito** numa caixa para tudo o que o seu cartão em Configuração oferece: iniciar ou parar, detalhes, instalar, entrar, modelos, atualizar, remover e mais.
- ▶ na caixa de um agente inicia-o num terminal dentro da janela.
- Os agentes ficam sempre no centro da parte visível do mapa enquanto desloca.
- O caminho ativo de cada agente — agente → o fornecedor que usa → o seu modelo — é desenhado a negrito. Enquanto um agente trabalha, uma luz percorre essa linha.
- A **caixa de estatísticas de uso**: enquanto um agente trabalha, pequenos números são escritos ao longo da sua linha até ao fornecedor, na cor da linha: quão rápido chegou a sua última resposta (**t/s**, tokens por segundo), o tamanho do seu contexto (**1M**) e quanto está em uso (**%**, amarelo a partir de 75 %, vermelho acima de 90 %, como as baterias). Vêm dos próprios ficheiros de sessão do agente (só números; Claude Code, Codex e Pi). O que mostra e o tamanho do texto ficam nas Definições.
- Uma linha vermelha ponteada liga um agente a um fornecedor para o qual está configurado mas que agora não usa (nenhum modelo dele em uso).
- Quando os agentes usam modelos diferentes de um mesmo LLM local, as linhas para esses modelos são violeta: mantém-nos carregados ao mesmo tempo ou alterna-os (ver Modelos ao mesmo tempo).
- A caixa de um agente online tem a mesma bateria que o seu cartão em Configuração: o que resta dos seus limites; a caixa de um LLM local em execução, a memória da GPU ainda livre.
- Quando um programa que não está no mapa usa um LLM local, a linha desse LLM até ao seu modelo fica grossa e numa cor própria (roxo ou ciano, a que se destacar das outras cores do tema), e uma luz percorre-a. Ver Usado por outras apps.
- Quando um agente iniciou outro — o Claude Code a executar `pi -p …` pela sua shell, por exemplo —, uma **seta** corre pela esquerda do cartão dos agentes, do agente que usa o outro até ao usado, com luz a percorrê-la; a sua dica nomeia ambos.
- O símbolo de atualizar (`F5`) na linha do cabeçalho volta a ver o que está instalado, configurado e com sessão iniciada, e repete as verificações do arranque (novas apps, add-ons, atualizações, problemas em aberto).
- Até a primeira linha ser desenhada, um cartão por cima do mapa mostra como em três passos; **Ver como** reproduz a animação da Ajuda. Depois da primeira ligação, ou com ×, desaparece para sempre.
- Zoom com **− 100 % +**, `Ctrl`+roda ou `Ctrl`+`+` / `-` / `0`.

> **DICA** Numa janela estreita (menos de cerca de 1300 píxeis, mais ou menos meio ecrã) o mapa desloca-se para o lado.

## 3.4 Problemas por resolver

Quando há algo a corrigir, aparece um **!** vermelho entre os símbolos do cabeçalho; a sua dica diz quantos, por exemplo *2 problemas por resolver · 4 sugestões*. Um clique abre **Problemas por resolver**: uma linha cada — um agente sem sessão iniciada, um LLM local sem modelos, uma ligação que não funciona, pouco espaço livre para modelos — com um botão até onde se corrige (os detalhes do

agente ou do LLM local, o mapa com essa linha realçada, Configuração, Arrumar). Os mesmos problemas aparecem como um **!** vermelho nos seus cartões em Configuração. As dicas e o que funciona estão nos próprios cartões e páginas de detalhes, e para que serve a configuração está em Configuração (**O que pode fazer com isto**). A verificação só lê; nunca altera, inicia ou envia nada, e não mostra nada enquanto o mapa está vazio.

## 3.5 Atualizações e sugestões

Por baixo dos problemas, na mesma página, **Atualizações e sugestões** lista o que há a dizer: programas ou skills novos neste PC, o que está neste PC mas não no mapa, e atualizações para os agentes e LLMs locais instalados, cada um com o seu botão (**Adicionar servidor MCP…**, **Configuração**, **Atualizar**). Quando só há estes e nenhum problema, o **!** do cabeçalho aparece na cor de destaque. As atualizações são procuradas em segundo plano ao abrir a janela, no máximo de seis em seis horas. A procura em si não instala nada; uma atualização só começa quando clica em **Atualizar**.

Com Definições › **Verificações IA diárias** ligado, as atualizações de servidores MCP que um agente encontrou na web também aparecem aqui, com **Página da versão** e **Fazer com …**; quando a nova versão já está neste PC, a linha oferece antes **Usar…** (ver Atualizações encontradas).

CAPÍTULO 4

# Agentes

## 4.1 Detalhes de um agente

Cada agente de programação tem um cartão em Configuração, os instalados com a sua versão. Clique no cartão de um agente instalado ( ASTERISM ou ··· › **Detalhes**), ou clique direito na sua caixa do mapa › **Detalhes**. No topo, uma linha diz se o agente está pronto (e se não, porquê) e se está a correr agora ou quando foi usado pela última vez. Por baixo, cada parte é um cartão que se recolhe: um clique no título abre-o ou fecha-o. Começam recolhidos, cada um com um resumo de uma linha e distintivos; uma parte que abra continua aberta nas páginas de detalhes seguintes.

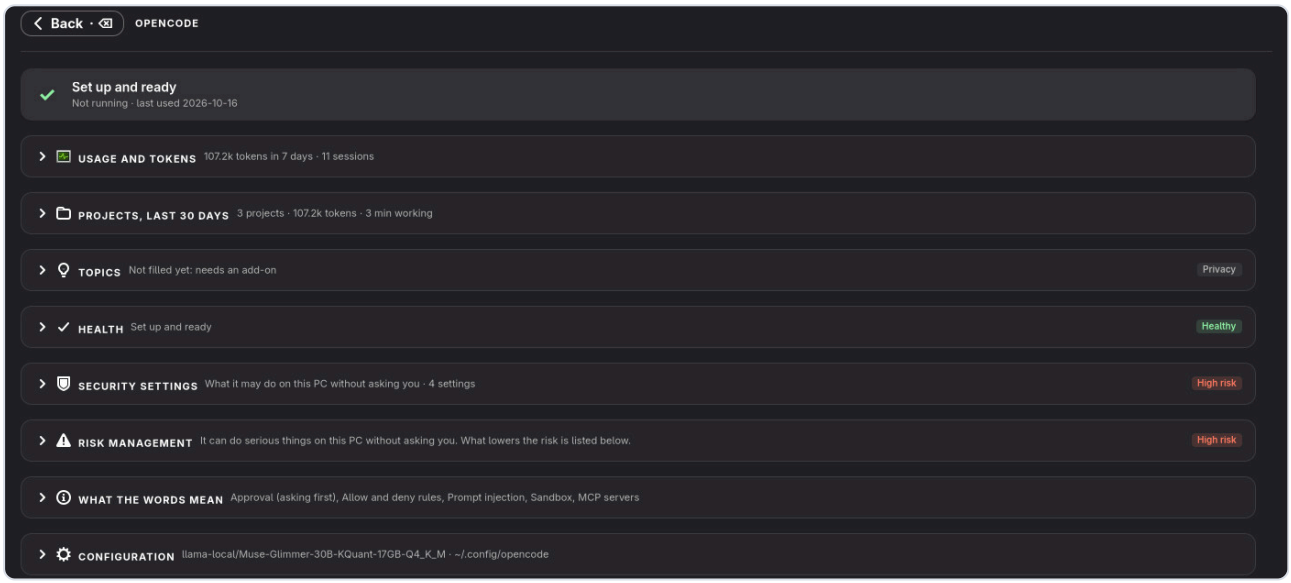



**Figura 5 ·** Os detalhes de um agente: cada parte recolhida, com o seu resumo e distintivos.

| Cartão | O que mostra |
|---|---|
| **Uso e tokens** | O que o agente regista: tokens e sessões de hoje e dos últimos sete dias, o plano, um gráfico de tokens por dia, limites com a hora de reposição e tokens por modelo. |
| **Projetos, últimos 30 dias** | Por pasta de projeto: tempo de trabalho, sessões, tokens, execuções de subagentes, linhas alteradas e o valor de API. |
| **Tópicos** (Privacidade) | Sobre o que eram as sessões. Vai além dos números, por isso está marcado com **Privacidade** e só um complemento o preenche (ver abaixo); o complemento dá nome ao cartão e o seu distintivo diz quanto lê. |
| **Estado** | Cada verificação da configuração com um ✓, ou o que fazer a seguir: entrar, instalar ou iniciar o seu LLM local, descarregar um modelo, adicionar servidores MCP ou skills. |

| Cartão | O que mostra |
|---|---|
| **Definições de segurança**, **Gestão de riscos**, **O que significam as palavras** | O que o agente pode fazer sem lhe perguntar e o que reduz o risco (ver Verificação de saúde e segurança). |
| **Configuração** | As definições do agente, servidores MCP, skills e a sua pasta de definições. |

Os projetos são mostrados para o Claude Code, o OpenCode e o Hermes: tempo de trabalho, sessões, tokens, execuções de subagentes, linhas alteradas e quanto custariam os tokens a preços de API. Os números vêm dos próprios ficheiros de sessão ou da base de dados de cada agente; nenhum texto é lido.

ASTERISM Com o complemento **Insights** (ver Complementos) o cartão **Tópicos** mostra primeiro que temas usaram mais tokens nos últimos 30 dias (**Mais tokens**: sessões com o mesmo título são um só tema, cada um com a sua parte de todos os tokens), depois os títulos que o agente deu às suas sessões, por projeto (**Por projeto**), e oferece **Resumir com o Claude…** (quando o Claude Code está instalado). Os seus tokens vêm dos próprios ficheiros de sessão do agente sem as leituras de cache, por isso podem ser muito mais baixos do que em **Uso e tokens**, que pode contar as leituras de cache. Antes de qualquer envio, vê exatamente o que vai ser enviado: os títulos e os números, nunca as conversas.

## 4.2 Iniciar e parar agentes

- ▶ na caixa do agente ou **Iniciar aqui** no seu menu inicia o agente num terminal dentro da janela, em vez do mapa. Precisa do VTE ( `vte4` ).
- **Iniciar no terminal** abre-o na sua própria janela de terminal.
- Enquanto corre dentro da janela: **Mostrar terminal** e **Parar**. Parar termina o agente e tudo o que ele iniciou, como os seus servidores MCP.

Quando há agentes com terminais dentro da janela, uma fila de separadores por cima do mapa mostra um separador por terminal, com um ponto para *a trabalhar*, *a correr* ou *terminado*. Um clique num separador mostra esse terminal; **×** fecha um terminado.

Um agente aberto em qualquer lado do computador recebe um ponto verde e *a correr*, e a caixa do modelo com que corre diz *em uso · <agente>*.

## 4.3 Tráfego de rede

A caixa e o cartão de um agente em execução mostram o que recebe e envia (↓ entrada, ↑ saída). Clique nessa linha, ou em ⇅ no cabeçalho para todos os agentes em execução, para abrir a sua página de rede: **Agora**, **Os últimos 5 minutos** como gráfico (entrada a verde, saída a azul), **Para onde vai** — API online, LLM locais, este PC e a sua rede local, cada um com a sua parte — e **Programas**: o próprio agente e os programas que ele iniciou, como os seus servidores MCP. É contado a partir das ligações TCP do próprio agente enquanto a app está aberta (sem direitos de administrador; o UDP não é contado) e fica neste PC.

## 4.4 Subagentes

O Claude Code e o OpenCode podem passar partes de uma tarefa a **subagentes**. Enquanto um dos subagentes de um agente trabalha, a sua caixa no mapa mostra uma pequena marca em árvore ao lado de ↑↓; se só tiver terminados na lista, a marca fica esmaecida. Clique nela para abrir a página **Subagentes** do agente: um mapa próprio, disposto como o grande, com o cartão do agente à esquerda e um cartão **Subagentes** à direita. Cada caixa mostra a tarefa do subagente, o seu tipo, a pasta onde a sua sessão trabalha e **a trabalhar** ou **concluído há … minutos**. A luz corre nas linhas para os que trabalham; os terminados ficam esmaecidos. Passe o ponteiro sobre uma caixa para ler toda a tarefa.

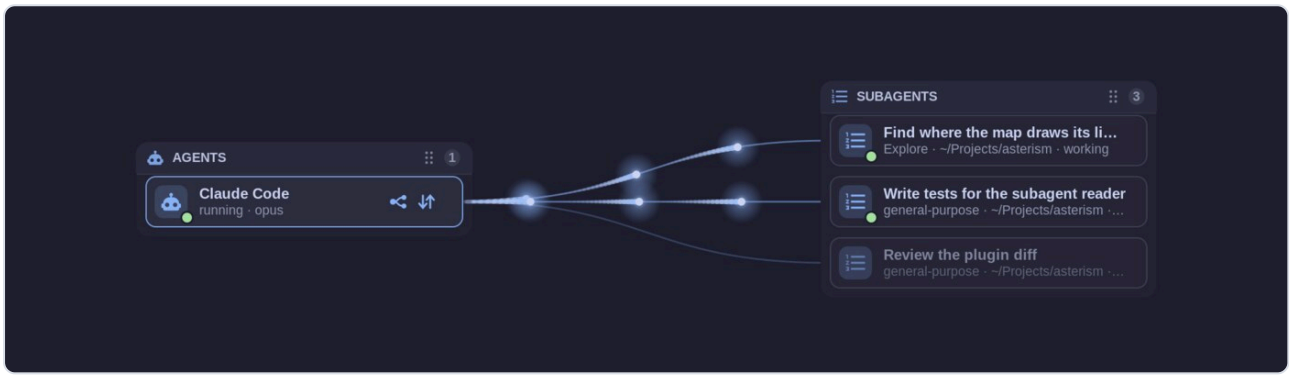



**Figura 6** · Os subagentes de um agente: dois a trabalhar, um concluído.

Quanto tempo os subagentes terminados ficam na lista (30 minutos) e quantos no máximo (10) define-se em Configurações › **Subagentes**; os que trabalham são sempre mostrados. A página só lê o tipo de cada subagente e a descrição de uma linha da sua tarefa, a partir dos ficheiros do próprio agente (as pastas de sessão do Claude Code, a base de dados do OpenCode), nunca o que os subagentes dizem.

## 4.5 Configurar…

Clique com o botão direito num agente ou num LLM local › **Configurar…** abre a sua própria configuração num separador de terminal da janela, para o que a app não define por si — por exemplo `hermes setup` ou o menu do Ollama; o LM Studio abre a sua janela. Um programa sem comando de configuração mostra o seu `--help`. Quando termina, o mapa volta a ver e mostra o que mudou, e a um LLM local em execução é proposto um reinício.

## 4.6 Entrar

**Entrar…** no menu de um agente (em Configuração ou no mapa) mostra o que o agente precisa e as suas próprias formas de iniciar sessão:

- **Uma subscrição ou uma conta** — o início de sessão do próprio agente corre num terminal dentro da janela (por exemplo `claude auth login` ou `codex login`). A sua palavra-passe ou o início de sessão no navegador ficam entre si e o agente; a app nunca os vê. Quando o agente o pede, o terminal diz o que escrever (Pi e Copilot: `/login`).
- **Uma chave de API** — para os agentes que leem uma: a chave vai para um campo de palavra-passe e daí diretamente para o próprio comando do agente (o Codex guarda-a ele mesmo) ou para o

ficheiro que o agente lê (Gemini: `~/.gemini/.env`, legível só por si). Nunca vai parar a uma linha de comandos, a um registo ou às definições da app, e a app não a guarda. A faturação por API do Claude Code é o seu próprio início de sessão na Anthropic Console, por isso nenhuma chave passa pela app.

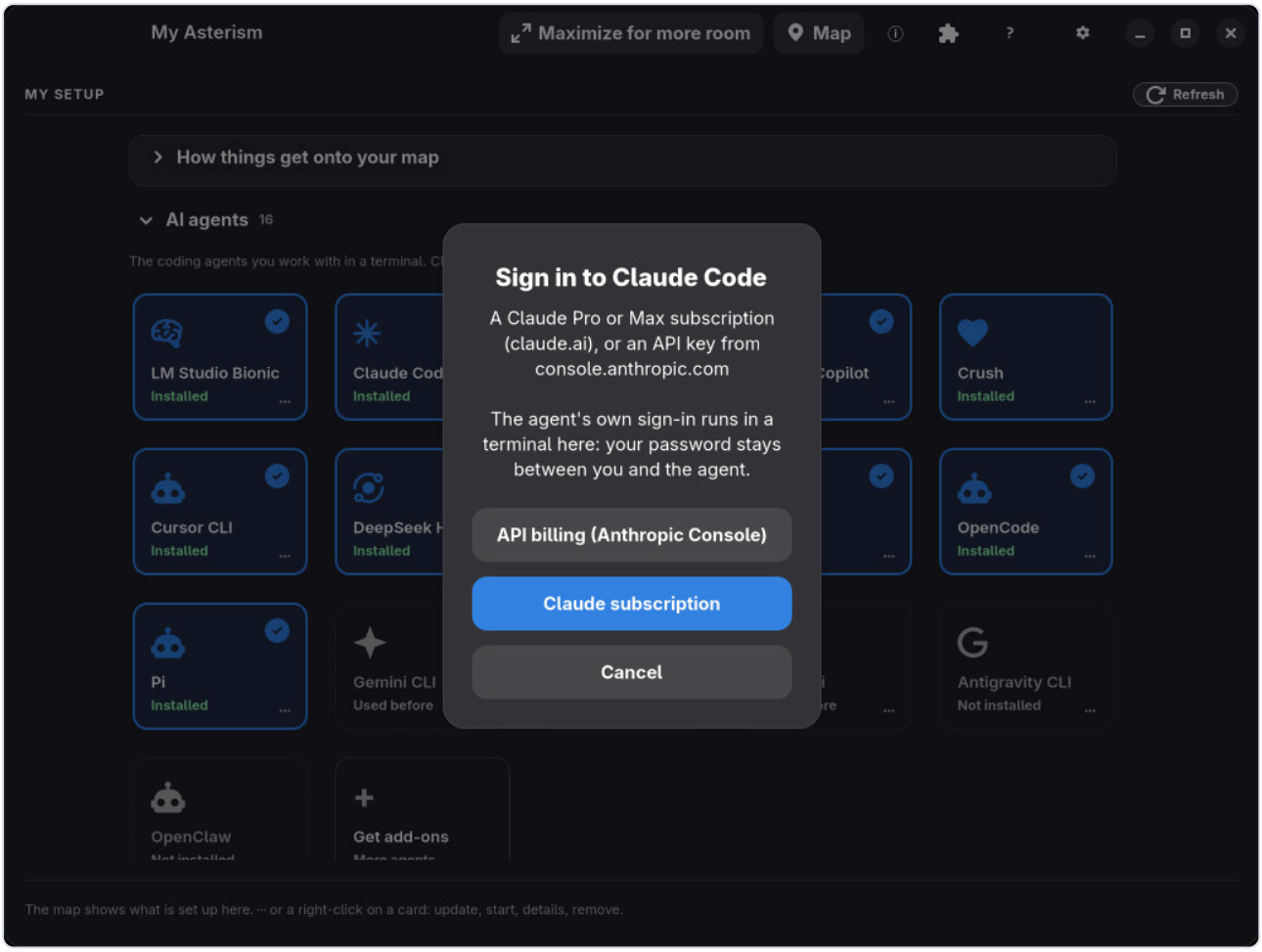



**Figura 7** · Entrar no Claude Code: uma subscrição ou faturação por API, ambas com o início de sessão do próprio Claude Code.

Depois, o mapa mostra o fornecedor online do agente e os seus modelos.

## 4.7 Atualizar agentes

Quando há uma versão mais recente de um agente instalado, o seu cartão em Configuração recebe um ↑ **versão** amarelo, e **Atualizações e sugestões** (o **!** do cabeçalho) menciona-a. **Atualizar** executa a atualização da mesma forma que o agente foi instalado — por exemplo `mise upgrade <ferramenta>` ou `claude update` — na sua própria janela de terminal, para que a possa acompanhar.

## 4.8 Agentes que a app pode instalar

Três agentes podem ser instalados, atualizados e desinstalados a partir do mapa (clique direito no agente › **Instalar...**):

- **DeepSeek Harness** (`dsh`, pré-visualização para programadores), a partir do npm. Pode usar LLMs locais.
- **Antigravity CLI** (`agy`, Google), a partir do manifesto de versões da Google; a transferência é verificada com a sua soma SHA-512.
- **LM Studio Bionic** não pode ser instalado daqui (sem soma de verificação publicada); obtenha-o em lmstudio.ai. Usa sempre o motor e os modelos do LM Studio.

## 4.9 Agentes novos

Quando aparece um agente que a app ainda não conhece, ela pede uma vez a um agente instalado (Claude Code, senão Codex) que descubra na web onde esse agente guarda as suas skills e servidores MCP. A resposta é verificada antes de ser usada, e nada dela é alguma vez executado. Definições › **Pesquisar agentes novos** desliga isto.

CAPÍTULO 5

# LLMs locais

Um motor de LLM local corre modelos de linguagem na sua própria placa gráfica, por isso os pedidos nunca saem do computador. A app instala-os, inicia-os e para-os por si.

## 5.1 Instalar um LLM local

Clique direito num LLM local cinzento (não instalado) no mapa › **Instalar...**. Escolhe a pasta do programa, a pasta dos modelos (é sugerida a que os seus outros LLMs locais usam) e se arranca ao iniciar sessão. É instalado só para o seu utilizador, com um serviço de utilizador, por isso não é preciso palavra-passe. Cada transferência tem de corresponder à soma SHA-256 publicada pela sua versão antes de ser desempacotada ou executada.

| Motor | Notas |
|---|---|
| **Ollama** | A versão oficial (com ROCm para placas AMD). |
| **llama.cpp** | A compilação que escolher (Vulkan, ROCm, CUDA, CPU), como encaminhador de modelos: qualquer GGUF na pasta dos modelos, um carregado de cada vez. |
| **LM Studio** | Não é instalado daqui (sem soma de verificação publicada); obtenha-o em lmstudio.ai. Uma vez instalado, é mostrado e gerido. |
| **vLLM** | Para placas NVIDIA, no seu próprio ambiente Python. |
| **Jan** | O seu AppImage, desempacotado, com o motor do Jan como serviço em segundo plano para os agentes. |
| **KoboldCpp** | A compilação oficial num só ficheiro, como encaminhador de modelos. |

Instalado assim, um LLM local oferece também **Atualizar...** (a versão anterior fica como recurso), **Pasta dos modelos...** e **Desinstalar...**, que remove o programa, o comando, o serviço e as definições, mas nunca os modelos. Os LLMs locais que instalou por si são mostrados, mas nunca alterados.

## 5.2 Iniciar e parar com os agentes

Ligue uma vez um agente a um LLM local e depois basta iniciar o agente — mesmo com a app fechada ou depois de reiniciar. O seu LLM local arranca com ele, e quando termina o último agente que o usa, o LLM para cerca de cinco segundos depois, ficando de novo livre a sua memória gráfica.

Disso trata um pequeno serviço em segundo plano, `ai-agents-map-providers.service`. É configurado com a primeira ligação a um LLM local e removido quando desaparece a última ou quando desativa Definições › **Iniciar LLMs locais com os seus agentes**.

> **NOTA** O KoboldCpp demora alguns segundos a arrancar, mais do que alguns agentes esperam. Por isso arranca ao iniciar sessão, e quando termina o último agente só o seu modelo é descarregado da memória.

## 5.3 Detalhes de um LLM local

Clique no cartão de um LLM local em Configuração, ou clique direito na sua caixa do mapa › **Detalhes**. Por baixo dos mosaicos (modelos, agentes, endereço) vêm os mesmos cartões recolhíveis dos agentes: **Uso e tokens** — o que os seus agentes enviaram através dele nos últimos dias, por agente e modelo, a partir dos próprios números de sessão dos agentes e associado pelo nome do modelo —, depois Estado, Gestão de riscos, O que significam as palavras e Configuração.

## 5.4 Memória da GPU e libertar memória da GPU

No cartão **GPU** por cima do mapa, **Memória da GPU** mostra quanta está em uso, repartida por LLMs locais, ambiente de trabalho e aplicações, e livre; **Atividade da GPU** mostra a carga, a temperatura e a potência. A caixa de cada LLM local mostra quanta memória ocupa, e os modelos carregados recebem um ponto verde.

Clique direito num LLM local para **Iniciar**, **Parar**, arrancar ao iniciar sessão e **Libertar memória da GPU**: os seus modelos são descarregados da memória, continua a correr e volta a carregar um quando lhe pedem.

Um LLM local que ocupa memória da GPU recebe um botão vermelho de parar na sua caixa. Ele pergunta antes: **Libertar memória da GPU** descarrega seus modelos; **Parar** o encerra. No LM Studio o botão é **Parar servidor**: só o servidor para, e a app continua aberta com os modelos carregados. Um modelo que um LLM local carregou, mas que nenhum agente usa, fica ligado a ele por uma linha clara contínua.

O LM Studio pode estar aberto sem o seu servidor. A sua caixa mostra então *servidor desligado* com um ponto amarelo, os agentes não o alcançam e o seu menu oferece **Iniciar servidor** (`lms server start`).

## 5.5 Modelos ao mesmo tempo

O llama.cpp, o LM Studio e o Ollama podem manter mais de um modelo carregado. Quando liga agentes a modelos diferentes de um mesmo LLM local, a app ajusta quantos ele mantém carregados ao mesmo tempo ao número que os seus agentes usam — com a mesma pré-visualização, cópia de segurança e Anular da ligação — e propõe reiniciá-lo se estiver em execução, porque ele lê esse número ao arrancar. Onde esse número não cabe à app (uma instalação do sistema ou o seu próprio comando de arranque), uma nota diz onde o definir.

Quando os modelos juntos não cabem na memória da GPU, um aviso di-lo: o LLM local alterna-os então, o que é lento, ou parte de um modelo corre no processador. Iniciar um agente verifica o mesmo: se carregar o seu modelo precisar de mais memória da GPU do que a livre, é-lhe perguntado antes, com o que ocupa a memória agora — **Libertar memória da GPU e depois iniciar**, **Iniciar mesmo assim** ou **Cancelar**.

## 5.6 Usado por outras apps

Os LLMs locais não são usados só por agentes. Um plugin como o Video Trans, uma app de chat ou o seu próprio script também podem enviar-lhes pedidos. A app repara nisso e mostra-o:




**Figura 8 ·** Um programa que não está no mapa (aqui `curl`) a gerar com o llama.cpp: a linha e a caixa ciano (tema Tokyo Night). O próprio caminho do Claude Code até ao opus fica na cor de destaque.

- **No mapa**, a linha do LLM local até ao modelo com que gera fica grossa e na sua própria cor, uma luz percorre-a, e a caixa do modelo recebe contorno e ponto nessa cor e diz quem o usa, por exemplo *em uso · Video Trans*.
- **O menu de clique direito do LLM local** e a sua página ⓘ **Detalhes** dizem *Em uso pelo Video Trans · agora* ou há quantos segundos foi usado pela última vez.
- A legenda ⓘ mostra ambas as marcas.

A app vê quais dos seus próprios programas têm uma ligação aberta ao LLM local. Nunca lê o que enviam. Um plugin aparece com o seu próprio nome, um agente com o seu, e tudo o resto com o nome do programa. A própria app e o LLM Monitor, que só leem o estado, não contam.

> **NOTA** A luz só se vê enquanto o programa gera e apaga-se cerca de dez segundos depois do seu último pedido. Um programa aberto mas inativo — o Video Trans sem fala para traduzir — não mostra nada. Um programa continua no menu meio minuto depois do seu último pedido.

CAPÍTULO 6

# Modelos

Clique direito num LLM local instalado › **Modelos…**, ou clique em **Adicionar modelo** em **Configuração**. A página tem uma lista à esquerda e o modelo escolhido à direita, em três separadores.

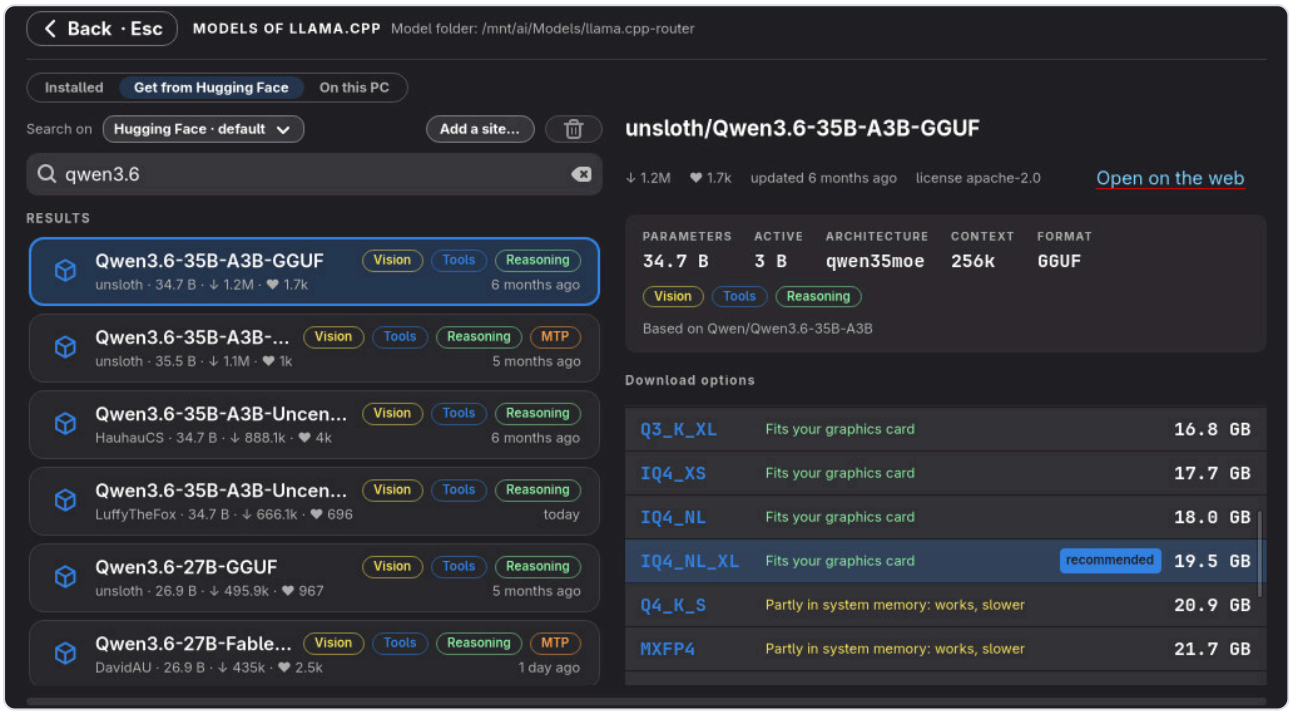



**Figura 9 ·** Obter do Hugging Face: o que um modelo sabe fazer e que tamanho cabe na sua placa gráfica.

## 6.1 Obter do Hugging Face

Explore os modelos de chat GGUF mais populares ou pesquise. Cada resultado mostra autor, tamanho, transferências, gostos, idade e o que sabe fazer: **Visão**, **Ferramentas**, **Raciocínio**, **MTP**. A página do modelo mostra parâmetros, arquitetura, contexto, licença e a ficha do modelo. **O que significa o nome** explica as partes do nome, como *35B-A3B* ou *Q4_K_M*.

**Opções de transferência** lista cada tamanho com o modo como cabe: na placa gráfica, em parte na memória (funciona, mas mais devagar) ou provavelmente demasiado grande. A melhor escolha já está feita; **Descarregar** traz-a para a pasta dos modelos.

As transferências correm em segundo plano e aparecem numa faixa no topo da página principal. Se a ligação cair, tentam de novo e continuam. Se a app foi fechada durante uma transferência, esta aparece como interrompida, com **Continuar** ou **Descartar**.

## 6.2 Instalados

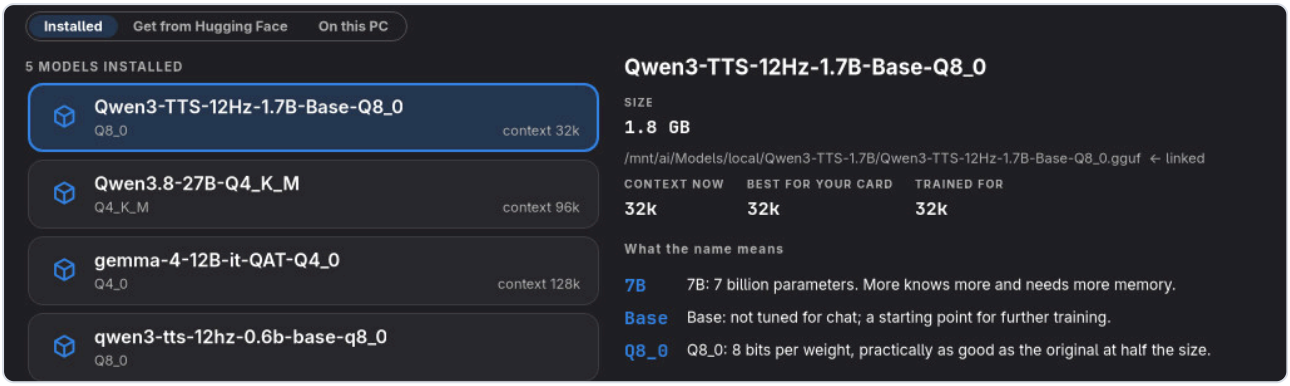



**Figura 10** · Instalados: contexto atual, o melhor para a sua placa e o do treino; o que significa o nome; Remover.

Cada modelo mostra tamanho, ficheiro e contexto — *atual*, *o melhor para a sua placa* e *treinado para*. **Remover** pergunta primeiro e move os ficheiros para o lixo, de onde os pode restaurar. Um modelo que é só uma ligação perde apenas a ligação.

## 6.3 Tamanho do contexto

O contexto é quanto texto o modelo consegue ter presente de uma vez. A app calcula, a partir do próprio ficheiro do modelo, o maior contexto que cabe na sua placa gráfica. **Definir contexto** escreve-o nas definições do LLM local, com cópia de segurança e Desfazer. Os agentes precisam de um contexto bastante grande; quando liga um agente a um modelo do LM Studio sem contexto definido, o melhor é definido no mesmo passo.

## 6.4 Neste PC

Lista ficheiros GGUF que o LLM local ainda não lê — na sua pasta dos modelos, nas pastas dos outros LLMs locais, em `~/Downloads` e em `~/Models`. **Adicionar** torna um deles modelo seu sem uma segunda cópia (através de uma ligação), exceto no Ollama, que importa a sua própria cópia.

## 6.5 Mover e eliminar modelos

Clique direito num modelo do llama.cpp, KoboldCpp ou LM Studio › **Mover para…**, ou arraste a caixa do modelo para a pasta de outro LLM local. Primeiro vê quanto é movido e que agente usa o modelo. Desfazer volta a mover os ficheiros. **Eliminar…** pergunta primeiro e move os ficheiros para o lixo.

## 6.6 Uma pasta dos modelos

Definições › **Pasta dos modelos** define uma pasta para os modelos de todos os LLMs locais. **Arrumar modelos…** mostra antecipadamente como os ficheiros seriam ordenados (`<pasta>/<editor>/<modelo>/`), e nada muda até premir **Arrumar**. Com **Os caminhos antigos continuam a funcionar** (ligado por omissão) fica uma ligação em cada local antigo. **Desfazer a última arrumação** repõe tudo.

> **ATENÇÃO** Ao mover para outro disco, os modelos são primeiro copiados e os originais só são removidos quando a cópia está completa. Certifique-se de que há espaço livre suficiente no destino; caso contrário, a app recusa a mudança.

CAPÍTULO 7

# Ligar desenhando

Prima numa caixa do mapa e arraste até outra para as ligar. Cada alteração mostra, antes de escrever, exatamente que linhas de que ficheiro mudam, faz uma cópia de segurança e pode ser desfeita a partir da barra que aparece a seguir.

## 7.1 Agente ↔ LLM local

Arraste de um agente para um LLM local ou ao contrário. Escolha o modelo e se o agente o usa logo. Para alterar uma ligação, selecione o seu agente (ou clique na linha): cada linha que pode ser movida ou removida mostra um **gancho laranja** na ponta. Arraste o gancho para outro LLM local — ou para o vazio para a remover; ao passar sobre um gancho vê o que faz arrastá-lo. Com um agente selecionado, um clique num modelo de um LLM local ligado torna esse modelo o modelo do agente.

Agentes suportados: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw e DeepSeek Harness.

## 7.2 Agente ↔ servidor MCP

Arraste de um agente para um servidor MCP. As definições do servidor são copiadas para o formato próprio desse agente. Suportados: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode e Pi. Os ficheiros de definições que podem conter chaves são sempre escritos de modo que só o utilizador os possa ler.

## 7.3 Agente ↔ pasta de skills

Arraste de um agente para uma pasta de skills. O Pi e o Hermes recebem a pasta nas suas definições; qualquer outro agente recebe uma ligação por skill na sua própria pasta de skills. Uma pasta que um agente lê por si (a maioria lê `~/.agents/skills`) não pode ser retirada só a esse agente.

## 7.4 Quando uma ligação falha

Logo depois de uma ligação ser escrita, a app verifica se vai funcionar: se um LLM local está instalado e oferece o modelo, se um servidor MCP responde, se as ligações das skills estão intactas. Uma ligação falhada fica no mapa como **linha vermelha tracejada** com um **!**; uma só com avisos é **amarela tracejada**. Clique na linha ou no seu **!** para ver sugestões:

- **Testar de novo** volta a verificá-la; se passar, a linha volta ao normal.
- **Tentar de novo…** abre a página de ligação para uma ligação que nunca foi escrita.
- **Remover ligação** remove-a.

CAPÍTULO 8

# Servidores MCP

Os servidores MCP dão ferramentas extra aos agentes. **+ Servidor MCP** no mapa ou por baixo dos cartões abre a página para adicionar: uma lista à esquerda, a página do servidor escolhido à direita.

## 8.1 De onde vêm os servidores

| Separador | Conteúdo |
|---|---|
| **Recomendados** | Os servidores de referência do projeto MCP (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) e servidores oficiais de empresas conhecidas (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), agrupados por finalidade e fixados na versão mais recente. |
| **Registo MCP** | Uma pesquisa no registo MCP oficial. Cada resultado indica o seu editor tal como o registo o verificou. A pesquisa pode demorar dez segundos ou mais. Os resultados vêm de 20 em 20; a linha por baixo da lista percorre-os e mostra onde está, por exemplo **‹ 20 anteriores** · 21 – 40 / 200 · **Próximos 20 ›** (com */ 40+* quando o registo não diz quantos há). |
| **Neste PC** | Programas neste computador que oferecem um servidor MCP, encontrados lendo os seus ficheiros — nada é iniciado. Só se olha para dentro dos AppImages quando clica em **Ver por dentro…**. |
| **À mão** | Uma linha de comandos ou um endereço web com cabeçalhos. |

## 8.2 Configurar um servidor

A página do servidor diz quem o fez e quão verificado está, o que faz (com coisas que pode pedir a um agente), como corre — como programa através de `npx`, `uvx` ou Docker, ou como serviço web — e do que precisa, como uma pasta ou uma chave de API com uma ligação para onde a obter. Por baixo, **Configurar**:

1. Preencha os campos de que o servidor precisa.
2. Assinale os agentes que o devem receber.
3. Clique em **Testar** para pedir ao servidor as suas ferramentas.
4. Clique em **Adicionar**: todos os agentes assinalados são escritos de uma vez, com pré-visualização e um Desfazer.

> **NOTA** Os valores secretos, como chaves de API, são passados no ambiente ou em cabeçalhos, nunca numa linha de comandos onde outros utilizadores do computador os possam ler. Um servidor que precisaria de um segredo na sua linha de comandos tem de ser configurado à mão.

## 8.3 Gerir servidores

ⓘ **Detalhes** no cartão de um servidor MCP mostra o ficheiro de definições de cada agente, o comando ou o endereço e, se pedir, as ferramentas do servidor; os valores das chaves nunca são mostrados. **Remover** retira-o de todos os agentes. Os servidores ficam na lista quando nenhum agente os usa, para que uma linha os possa voltar a ligar.

Quando aparece em `~/Applications` uma versão mais recente de uma app que fornece um servidor MCP, uma sugestão diz isso. **Atualizar…** desempacota a nova versão, aponta cada agente para ela e guarda a antiga sete dias como cópia de segurança. **Procurar atualizações** (em **Neste PC**) faz isto para todas essas apps de uma vez depois do seu clique: olha para dentro das novas versões, oferece todas as atualizações numa pré-visualização com uma cópia de segurança e um Desfazer, e depois testa cada servidor atualizado; uma sugestão indica qualquer um que não responda.

CAPÍTULO 9

# Skills

Uma skill é uma pasta de instruções que ensina a um agente uma tarefa concreta. **+ Skills** abre uma página organizada como a dos MCP.

## 9.1 Encontrar skills

- **Neste PC**: pastas de skills que as apps trouxeram e que nenhum agente lê ainda.
- **Da web**: uma pesquisa no diretório skills.sh, noutro site com a mesma API ou num repositório GitHub de skills. Uma skill é obtida do seu repositório GitHub para uma pasta à sua escolha, por omissão `~/.agents/skills`.

> **ATENÇÃO** As skills podem conter scripts que um agente pode executar mais tarde. A página mostra o repositório e lista os scripts antes de obter o que quer que seja — veja-os.

## 9.2 Pastas de skills

Cada cartão de pasta de skills mostra as suas skills e os agentes que a leem. Uma pasta partilhada através de ligações aparece uma só vez, onde realmente está. ⓘ **Detalhes** lista cada skill com a sua descrição e como cada agente recebe a pasta.

CAPÍTULO 10

# Definições e ajuda

## 10.1 Definições

A roda dentada (`Ctrl`+`,`) abre as definições. `Retrocesso` ou `Esc` volta atrás.

| Definição | O que faz |
|---|---|
| **Idioma** | O idioma do sistema ou English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski. |
| **Aparência** OMARCHY | As cores do tema Omarchy ou a aparência Adwaita do GNOME com o modo claro/escuro e a cor de destaque do sistema. |
| **Animações do mapa** | Uma luz percorre as linhas do mapa: **Agentes e outros programas** (a linha de um agente a trabalhar e a linha de um LLM local até um modelo que outro programa usa), **Só os meus agentes** ou **Desligado**. Um pequeno mapa animado por baixo da definição mostra o que cada escolha move. |
| **Caixa de estatísticas de uso** | Os números ao longo da linha de um agente a trabalhar (ver Trabalhar com o mapa): ligada ou não, quais de **Tokens por segundo**, **Tamanho do contexto** e **Contexto usado** mostra, e o seu **Tamanho do texto** (7 pt por predefinição). |
| **Subagentes** | Para a página Subagentes: quanto tempo ficam os subagentes terminados (**30 minutos** por omissão; 10, 30, 60 ou 120) e quantos no máximo (**10** por omissão; 5, 10, 20 ou 40). Os que trabalham são sempre mostrados. |
| **Separadores de terminal acima do mapa** | Um separador por cada terminal de agente dentro da janela. |
| **Cantos** | Como as janelas, arredondados ou retos. |
| **Iniciar LLMs locais com os seus agentes** | O vigilante em segundo plano de Iniciar e parar com os agentes. |
| **No lançador** | A entrada do lançador; desligado remove-a. |
| **Os agentes arrancam em** | A pasta em que os agentes arrancam. |
| **Funções de IA**: **Pesquisar agentes novos** | Perguntar a um agente instalado onde um agente desconhecido guarda as suas definições (ligado por omissão). |
| **Funções de IA**: **Verificações IA diárias** | Uma vez por dia, um agente procura na web novas versões dos seus servidores MCP (desligado por omissão; usa os tokens do agente). Ver Smart Check. |
| **Funções de IA**: **Agente para as verificações IA** | Claude Code ou Codex: faz as verificações IA e as pesquisas e assume as tarefas **Fazer com …** e **Corrigir com …**. |

| Definição | O que faz |
|---|---|
| **Pasta dos modelos** | Uma pasta para todos os modelos, com **Arrumar modelos…**. |
| **Complementos** ASTERISM | **Procurar complementos online**, a página de complementos e **Configuração**. |
| **Sobre** | A versão, o autor e a licença; **Site** abre o projeto no GitHub e **Comunicar um problema** a sua lista de problemas. |

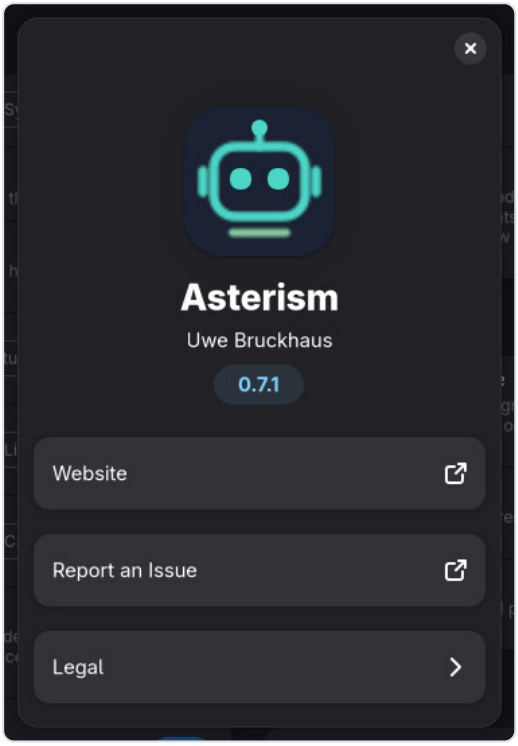



**Figura 11** · Definições › Sobre.

## 10.2 Ajuda

**?** (F1) abre a ajuda: **Abrir o manual** abre este manual (a edição que veio com a sua versão, no seu idioma) no seu visualizador de PDF; **Espaço em disco** (abaixo); uma animação sobre configurar desenhando; e o LLM Monitor, um plugin de barra complementar que mostra o que um LLM local está a fazer (uma pequena animação da sua entrada na barra e do seu painel). Ambas as animações são desenhadas nas cores do seu tema e seguem de imediato uma mudança de tema. A legenda ⓘ do cabeçalho explica cada ponto, linha e marca do mapa.

O **i** preenchido no cabeçalho abre **Informações**: cartões dos outros plugins do autor, lidos do Omarchy Plugin Marketplace quando a página abre, no máximo uma vez por hora (sem rede, a lista que esta versão conhece). Um clique num cartão abre a página do plugin no marketplace.

## 10.3 Espaço em disco

**Espaço em disco: agentes, LLM locais e modelos** mostra quanto espaço ocupa a sua configuração de IA e onde. É medido em segundo plano sempre que a ajuda abre (de novo após dez minutos ou com **Medir de novo**) e só lido — nada é apagado daqui. O título mostra sempre os totais numa linha (*Agentes · LLMs locais · Modelos · No total*); um clique recolhe a secção. **Discos**, **Agentes**, **LLMs locais** e **Pastas de modelos** começam recolhidos, cada um com o seu total e os itens maiores; um clique abre-os.

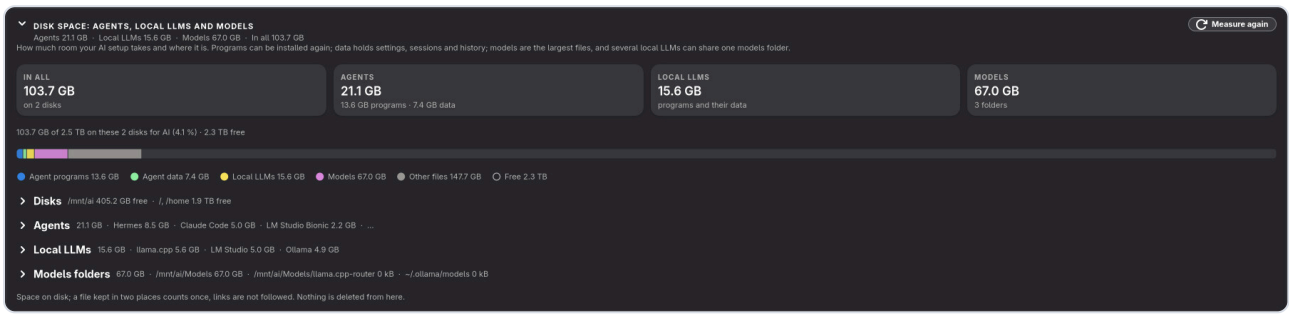



**Figura 12** · Os totais, a parte da IA em todo o espaço em disco e as secções recolhidas com os seus totais.

- **Mosaicos**: tudo junto, depois programas dos agentes, dados dos agentes (definições, sessões, histórico), LLMs locais (programas e os seus próprios dados) e modelos.
- **Barra**: todo o espaço dos discos onde está a configuração de IA. As partes coloridas são a configuração de IA, o cinzento é tudo o resto nesses discos e o resto vazio está livre; a linha por cima dá a parte da IA em percentagem. Passe o rato pela barra para ver os tamanhos exatos.
- **Discos**: cada disco com o que a configuração de IA ocupa nele e quanto está livre.
- **Agentes**, **LLMs locais**, **Pastas de modelos**: uma linha cada, os maiores primeiro. Abra uma linha para ver cada pasta com tamanho e número de ficheiros e abri-la no gestor de ficheiros. Uma pasta de programa diz quantas versões são guardadas; para um agente com várias versões antigas no mise, uma dica menciona `mise prune`, que remove as que não são usadas. Uma pasta de modelos diz que LLMs locais a usam, ou que só contém ligações para modelos de outra pasta.

> **DICA** Os programas podem ser reinstalados e os modelos descarregados de novo; os dados dos agentes contêm as suas sessões e definições. Quando o espaço escassear, olhe primeiro para versões antigas de programas e para modelos que já não usa.

Os tamanhos são espaço em disco: um ficheiro guardado em dois sítios conta uma vez, e as ligações não são seguidas.

CAPÍTULO 11

# Smart Check

O Smart Check deixa um agente online instalado — Claude Code ou Codex — examinar a configuração de IA deste PC, encontrar atualizações para os seus servidores MCP e fazer correções por si, e mostra quanto isso custou em tokens. As duas estrelinhas do cabeçalho abrem-no; só aparecem enquanto um destes agentes estiver instalado. Definições › **Agente para as verificações IA** escolhe qual.

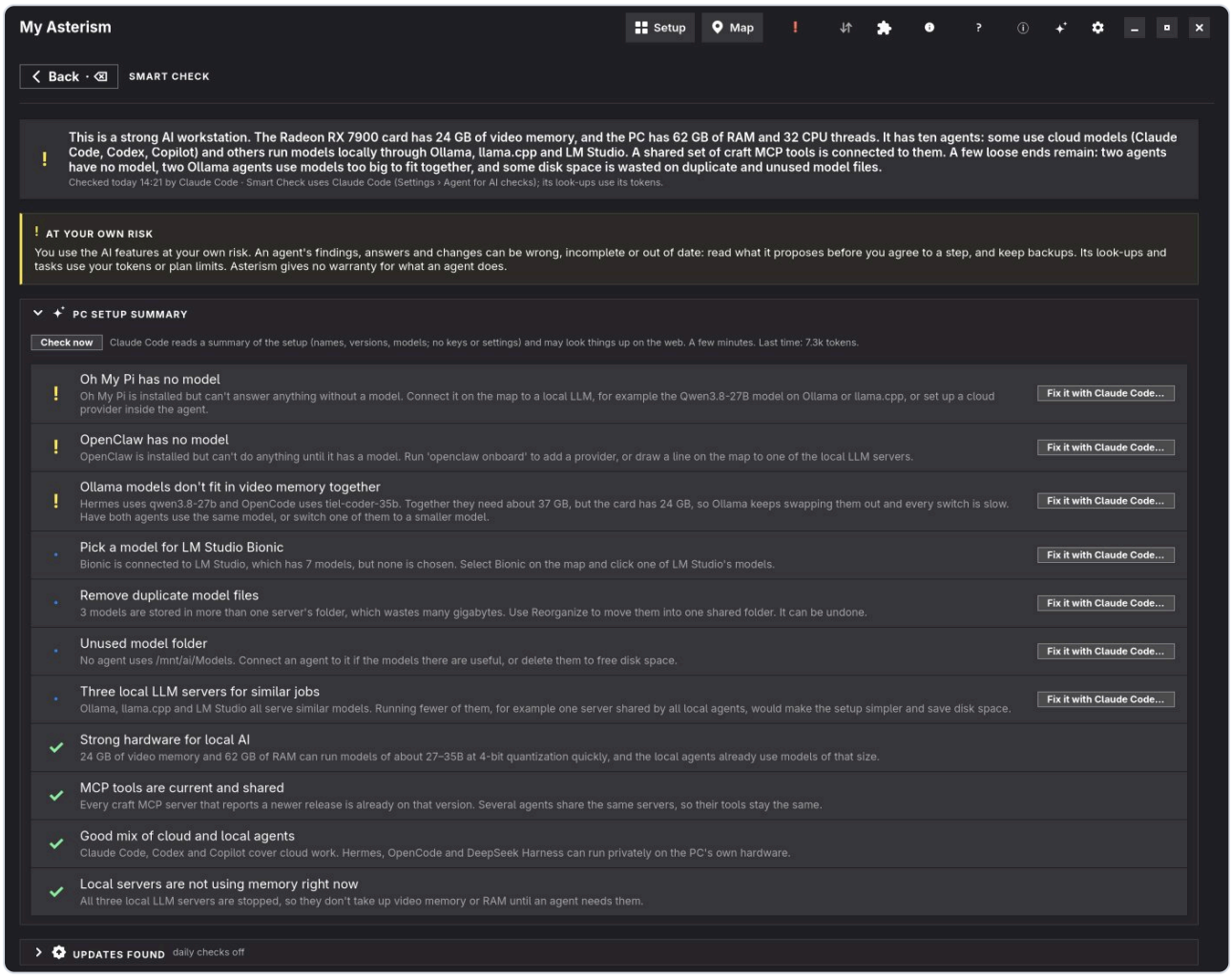



**Figura 13** · O Smart Check depois de uma verificação pelo Claude Code: problemas (!), dicas (·) e o que está bem (✓).

> **POR SUA CONTA E RISCO** As conclusões, respostas e alterações de um agente podem estar erradas, incompletas ou desatualizadas. Leia o que propõe antes de aceitar um passo e mantenha cópias de segurança. As suas pesquisas e tarefas usam os seus tokens ou os limites do seu plano. A app não dá qualquer garantia sobre o que um agente faz. A mesma nota encabeça Definições › **Funções de IA**.

## 11.1 Resumo da configuração do PC

**Verificar agora** dá ao agente um pequeno texto sobre a configuração — a placa gráfica e a memória, os agentes com os seus modelos, os LLMs locais, os servidores MCP com as suas versões, as skills e o que Problemas por resolver encontrou — e deixa-o procurar coisas na web. Não contém chaves, cabeçalhos nem valores de definições. Demora um minuto ou mais em segundo plano; a linha por baixo do botão diz quantos tokens a última verificação usou. A resposta é um pequeno resumo e uma lista de resultados: **!** problemas, • dicas e ✓ o que está bem. As respostas que não se confirmam são deixadas de fora.

**Corrigir com …** ao lado de um problema ou de uma dica abre o agente num separador de terminal dentro da janela, com esse resultado como tarefa: verifica se é mesmo assim, explica e pergunta-lhe antes de cada passo que altere alguma coisa. Passar servidores MCP para uma nova versão fica a cargo do **Atualizar** da app (abaixo), que altera as definições de todos os agentes de uma vez, com cópia de segurança e Desfazer.

## 11.2 Atualizações encontradas

Novas versões dos seus servidores MCP, de duas fontes:

- **As verificações IA diárias** (Definições › **Verificações IA diárias**, desligado por omissão): uma vez por dia o agente procura nas páginas oficiais de versões novas versões de servidores cuja versão a app consegue saber — apps descompactadas em `~/Applications/<app>-<version>/` e pacotes npm ou PyPI fixados numa versão. **Verificar agora** neste cartão pergunta de imediato, também enquanto as verificações diárias estão desligadas. Cada resultado tem **Página da versão** e **Fazer com …**: o agente descarrega o novo AppImage num separador de terminal, perguntando antes de cada passo.
- **A própria procura da app**: uma versão mais recente já está neste PC, mas um agente ainda inicia a antiga (o mesmo que a atualização no cartão do servidor em Configuração).

Quando a nova versão está neste PC, a sua linha di-lo e oferece **Usar…**: a pré-visualização da atualização mostra que agentes passam para ela; **Atualizar** altera-os todos, guarda a versão antiga com a cópia de segurança e testa cada servidor a seguir. Um servidor cujos agentes já iniciam todos a nova versão sai da lista.

> **DICA** Depois de uma ação iniciada aqui — uma atualização ou uma tarefa num separador de terminal —, **Voltar** leva-o de novo ao Smart Check, atualizado.

## 11.3 Estatísticas de tokens

Tokens usados hoje e nos últimos 30 dias, um gráfico por dia e os totais por agente e por plugin (outros plugins Omarchy que usam o mesmo assistente de IA também aparecem aqui). **Valor em API** é quanto as execuções custariam a preços de API, e a linha por baixo diz se a sua subscrição o cobre ou se é cobrado a uma chave de API. Uma tarefa num separador de terminal conta na sessão própria do seu agente; a página de detalhes do agente mostra esses tokens.

Mais abaixo, um cartão por tipo de tarefa — a verificação da configuração, atualizações de servidores MCP, a pesquisa de agentes novos, correções — com as suas execuções, tokens e os resultados mais recentes.

CAPÍTULO 12

# Verificação de saúde e segurança

Um agente de programação lê e escreve ficheiros e executa comandos no seu computador. Quanto pode fazer *sem lhe perguntar* está nas suas próprias definições, com palavras diferentes em cada agente. A **Verificação de saúde e segurança** junta tudo isto: na página de detalhes de cada agente e de cada LLM local, quatro cartões mostram se funciona, o que pode fazer, o que isso significa em palavras simples e o que reduz o risco.

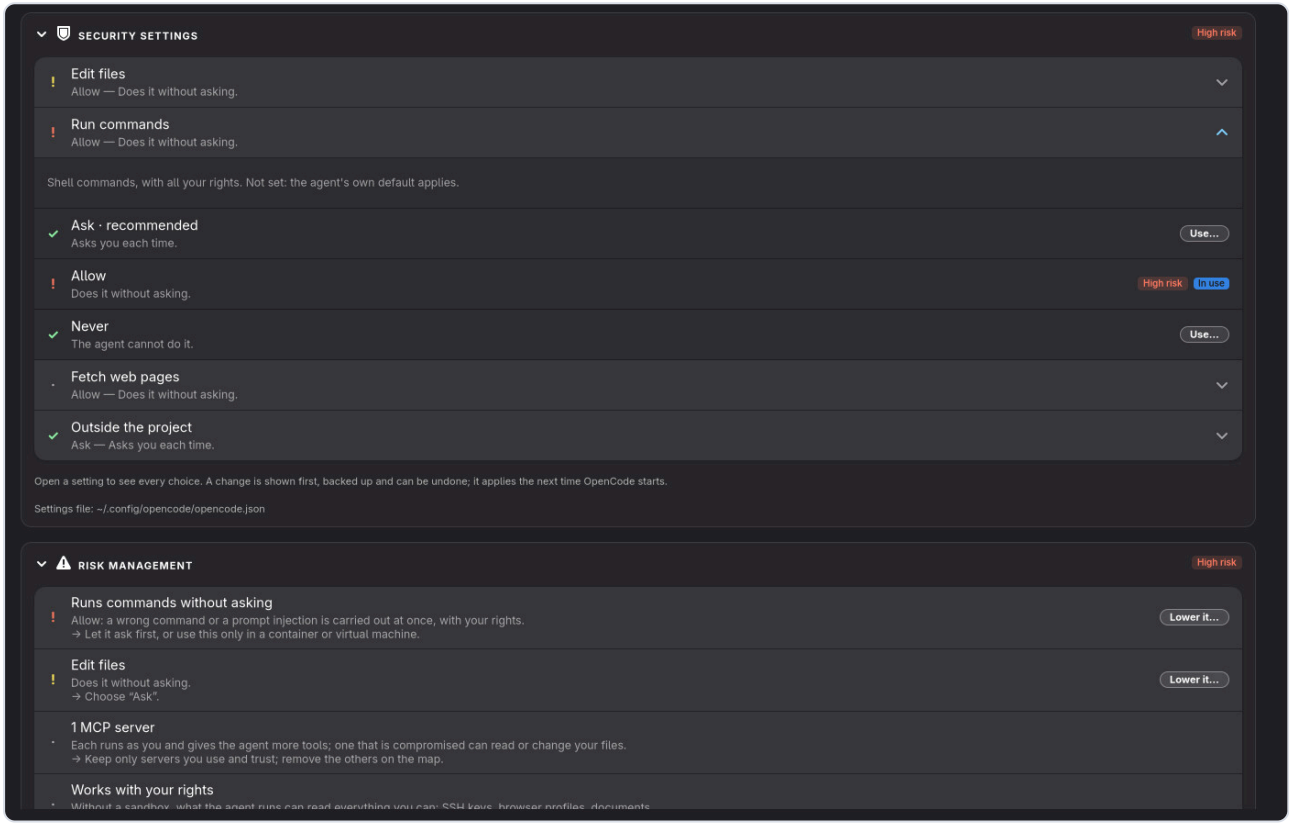



**Figura 14** · As definições de segurança do OpenCode com **Executar comandos** aberto, e o início da sua gestão de riscos.

## 12.1 Ler os cartões

Os distintivos dos cartões recolhidos resumem tudo: **Saudável**, **Precisa de atenção** ou **Não configurado** em Estado, e **Risco baixo**, **Risco médio** ou **Risco alto** em Definições de segurança e Gestão de riscos. Abra um cartão para ver os pormenores:

| Cartão | O que mostra |
|---|---|
| **Estado** | Cada verificação da configuração com um ✓, ou o que fazer a seguir: instalado, com sessão iniciada, o seu modelo e LLM local, servidores MCP e skills. |
| **Definições de segurança** | As definições do agente que decidem o que pode fazer sozinho, cada uma com o valor atual e uma frase sobre o que significa. O sinal à frente indica o risco: ✓ seguro, · baixo, um **!** amarelo médio, um **!** vermelho alto. |

| Cartão | O que mostra |
|---|---|
| **Gestão de riscos** | O que as definições significam em conjunto, o mais sério primeiro, cada um com o motivo e → o que o reduz. As proteções existentes aparecem com um ✓. |
| **O que significam as palavras** | Sandbox, aprovação, regras de permitir e negar e as outras palavras dos cartões, explicadas (ver As palavras explicadas). |

As regras de permitir e negar e as pastas de confiança do agente (por exemplo a lista `permissions.allow` do Claude Code) aparecem por baixo das definições, uma regra por linha.

## 12.2 Alterar uma definição

1. Clique numa definição para a abrir. Vê o que controla e cada escolha com o seu risco; a que está em uso tem a marca **Em uso**, a escolha mais segura **recomendado**.
2. Clique em **Usar …** junto a uma escolha ou em **Reduzir …** junto a um risco. Uma página mostra a alteração em palavras (antes → depois), o risco antes e depois, e **O que muda**: as linhas do ficheiro de definições do agente.
3. Clique em **Alterar**. O ficheiro é copiado em segurança e escrito; uma notificação oferece **Desfazer** durante 30 segundos. O agente usa a nova definição no próximo arranque.

> **NOTA** Só essa chave muda; os comentários e tudo o resto no ficheiro ficam como estão. Uma definição que precisa de mais de uma linha num ficheiro — como o Hermes a executar os seus comandos em Docker — é mostrada e explicada, com onde a alterar, mas não é alterada daqui.

## 12.3 O que é verificado em cada agente

| Agente | Definições mostradas e alteráveis | Ficheiro |
|---|---|---|
| **Claude Code** | Modo de permissões (Perguntar primeiro, Só planear, Só o permitido, Editar sem perguntar, Revisão automática, Saltar todas as verificações), sandbox para comandos, bloquear o modo de contorno; regras permitir/perguntar/negar | `~/.claude/settings.json` |
| **Codex** | Quando o Codex pergunta, sandbox (só leitura, pasta do projeto, nenhuma), rede na sandbox; pastas de confiança | `~/.codex/config.toml` |
| **GitHub Copilot** | Modo de permissões (Perguntar primeiro, Revisão automática, Permitir tudo); pastas de confiança, sites sem perguntar | `~/.copilot/settings.json` |
| **Cursor CLI** | Aprovação, sandbox, rede na sandbox, pesquisa web sem perguntar; regras permitir/negar | `~/.config/cursor/cli-config.json` |
| **Hermes** | Comandos perigosos (perguntar sempre, revisão automática, nunca perguntar), ocultar segredos; onde correm os comandos (só mostrado) | `~/.hermes/config.yaml` |

| Agente | Definições mostradas e alteráveis | Ficheiro |
|---|---|---|
| **OpenCode** | Perguntar, permitir ou nunca para editar ficheiros, executar comandos, obter páginas web e trabalhar fora do projeto | `~/.config/opencode/opencode.json` |
| **Pi** | As suas ferramentas: ler, escrever e executar comandos, ou só ler. O Pi, por conceção, nunca pergunta, por isso as suas ferramentas são as suas permissões. | `~/.pi/agent/settings.json` |

Para outros agentes, o cartão mostra o seu estado e os seus servidores MCP e diz onde procurar nas próprias definições do agente. Para um **LLM local** mostra quem o pode alcançar: só este PC ( `127.0.0.1` , como deve ser) ou qualquer dispositivo da sua rede.

## 12.4 Controlar o risco

O nível de risco é o ponto mais sério que o cartão encontra:

- **Risco alto** — pode fazer coisas sérias sem perguntar: executar qualquer comando, ou trabalhar sem sandbox e sem perguntas. Aceitável num contentor descartável ou numa máquina virtual; no seu próprio PC, faça-o perguntar primeiro.
- **Risco médio** — algumas coisas acontecem sem perguntar, como editar ficheiros do projeto, ou uma revisão automática decide por si.
- **Risco baixo** — pergunta antes de tudo o que importa, ou não o pode fazer de todo.

Bons hábitos, sejam quais forem as definições:

- **Perguntar primeiro nos comandos** no seu próprio computador; permita comandos isolados e inofensivos (como `git status` ) com uma regra de permitir em vez da shell inteira.
- **Ligue a sandbox** onde o agente tiver uma (Claude Code, Codex, Cursor CLI) e deixe a rede lá dentro desligada a não ser que uma tarefa precise dela.
- **Inicie os agentes na pasta do projeto**, não na sua pasta pessoal, e mantenha chaves e palavras-passe longe.
- **Poucos servidores MCP**: só os que usa e em que confia. Cada um corre com as suas permissões.
- **O trabalho sem supervisão** (modo de contorno, «YOLO», Pi com todas as ferramentas) deve correr num contentor ou numa máquina virtual.
- **LLMs locais em `127.0.0.1`** : os seus servidores não têm palavra-passe.

> **ATENÇÃO** Uma revisão automática (o modo *auto* do Claude Code, *assisted* do Copilot, *auto-review* do Cursor, *smart* do Hermes) é uma segunda IA que julga cada passo. Poupa perguntas, mas é um juízo, não uma muralha.

## 12.5 As palavras explicadas

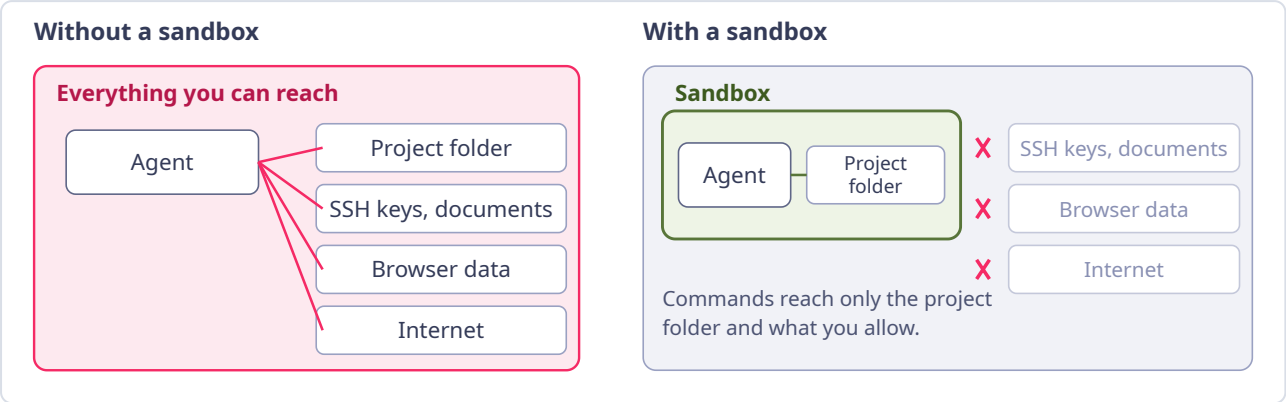



**Figura 15** · O que muda uma sandbox: os comandos só alcançam a pasta do projeto e o que permitir.

| | |
|---|---|
| **Sandbox** | Um espaço isolado onde correm os comandos do agente. Lá dentro, um comando só pode tocar na pasta do projeto (e no que permitir) — não no resto da sua pasta pessoal, nas suas chaves SSH ou nos dados do navegador, e muitas vezes nem na internet. Sem sandbox, um comando do agente pode fazer tudo o que o utilizador pode. |
| **Aprovação (perguntar primeiro)** | Antes de o agente editar um ficheiro ou executar um comando, mostra o que quer fazer e espera pelo seu sim ou não. Quanto menos puder fazer sem perguntar, mais controlo mantém — e mais vezes lhe pergunta. |
| **Regras de permitir e negar** | Listas de comandos ou ferramentas que o agente pode usar sem perguntar (permitir) ou nunca (negar). Negar ganha a permitir. Uma regra que permite qualquer comando de shell desliga a aprovação para a shell. |
| **Pasta de confiança** | Uma pasta em que disse que o agente pode trabalhar. Lá, usa também as próprias instruções e definições da pasta, que podem vir de outra pessoa quando clona um projeto. |
| **Acesso à rede** | Se os comandos podem alcançar a internet. Código que lê os seus ficheiros e pode enviar dados poderia divulgá-los; instalar pacotes também precisa da rede. |
| **Contorno («YOLO»)** | Tudo sem perguntar e sem limites. Só para uma VM ou um contentor descartáveis. |
| **Revisão automática** | Um segundo modelo de IA deixa passar as ações inofensivas e pergunta pelas restantes. |
| **Injeção de prompts** | Instruções escondidas numa página web, num ficheiro ou na resposta de uma ferramenta, que tentam levar o agente a fazer algo que não pediu. Perguntar primeiro, uma sandbox e nenhuma rede travam-nas. |
| **Ocultar segredos** | Texto que parece uma palavra-passe, um token ou uma chave é mascarado antes de o modelo o ver ou de ir parar a um registo. |
| **Ferramentas** | O que o agente pode fazer de todo: ler, escrever, executar comandos, pesquisar na web. Uma ferramenta que não tem não pode ser mal usada. |
| **Servidor MCP** | Um pequeno programa que dá ferramentas extra ao agente. Corre como o utilizador, com as suas permissões. |
| **127.0.0.1 (localhost)** | Um endereço que só os programas deste PC alcançam. Um servidor em `0.0.0.0` é alcançável a partir de qualquer dispositivo da sua rede. |

CAPÍTULO 13

# Complementos

Tudo o que o Asterism entende de um agente ou de um fornecedor de LLM é um **adaptador**: como é encontrado e instalado, as suas sessões, definições de segurança, servidores MCP e skills. No Asterism, cada adaptador é um **complemento**: os pacotes trazem-nos todos, assinados e desativados, e o utilizador ativa os que usa. Os complementos também trazem agentes novos, fornecedores novos e funções que vão além dos números, como o **Insights** (tópicos das sessões e um resumo do Claude). O plugin AI Agents Map tem os adaptadores integrados e não tem complementos.

## 13.1 Configurar o Asterism

Um Asterism novo abre na sua **página de boas-vindas**. Cada agente e LLM local que o Asterism conhece é um cartão com o seu próprio ícone, em três grupos: **Agentes de IA**, **LLMs locais** e **Fornecedores online** (os nomes da Anthropic, da OpenAI e dos outros no mapa). Cada cartão diz se o programa está aqui:

- **Instalado** (verde) — encontrado da mesma forma que o mapa o encontra; o cartão já está assinalado;
- **Usado antes** — agora não instalado, mas as suas definições ainda lá estão;
- **Não instalado** — assinale-o para o instalar depois a partir do seu cartão.

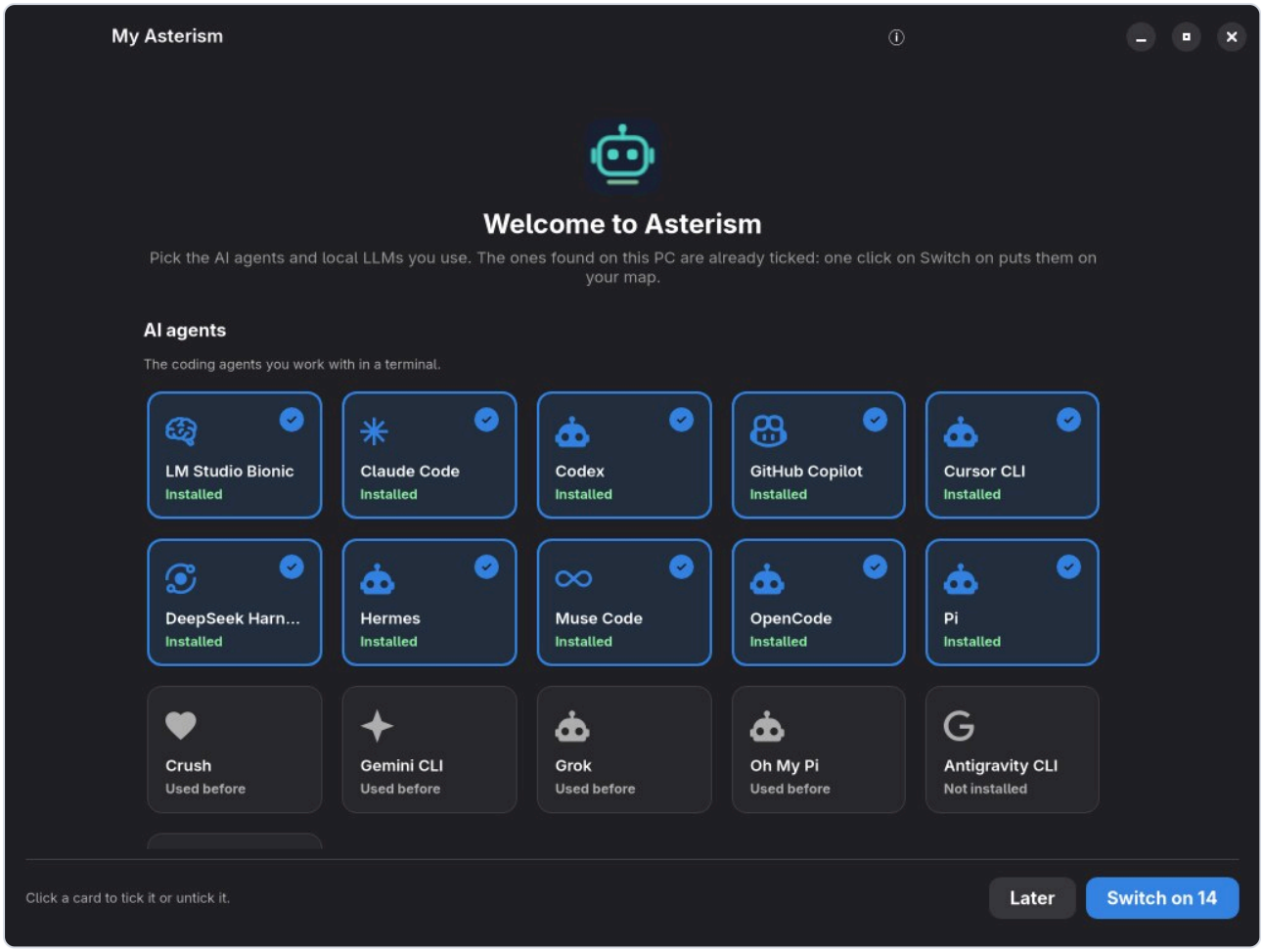



**Figura 16** · A página de boas-vindas: os programas encontrados vêm primeiro e estão assinalados.

Clique num cartão (ou prima `Espaço` sobre ele) para o assinalar ou desassinalar; um cartão assinalado tem contorno azul e um visto. **Ativar** (com o número de cartões assinalados) carrega-os e mostra o mapa; se assinalou programas que não estão instalados, abre-se **Configuração**: ··· nos seus cartões › **Instalar...**. **Mais tarde** começa com um mapa vazio. Enquanto nada estiver ativado, o Asterism volta a abrir na página de boas-vindas em cada arranque. Depois, os mesmos cartões estão em **Configuração** (o botão do cabeçalho), onde agentes e fornecedores se ativam e desativam a qualquer momento.

## 13.2 A página de complementos

Abra-a com o botão da peça de puzzle no cabeçalho, a partir de Definições › **Complementos** ou de qualquer sugestão «complemento disponível». Mostra, em cartões de grupo recolhíveis que começam abertos:

- **Sugeridos para este PC** — um complemento cujo agente ou fornecedor está instalado aqui mas ainda não é mostrado;
- **Complementos instalados** — ativado, desativado ou *não carregado* com o motivo (por exemplo um ficheiro alterado); agentes e fornecedores separados por uma linha;
- **Disponíveis** — os restantes complementos do catálogo;
- **Incluído no Asterism** — agentes e fornecedores que nenhum complemento instalado traz (quando corre o Asterism a partir da sua pasta de código sem complementos).

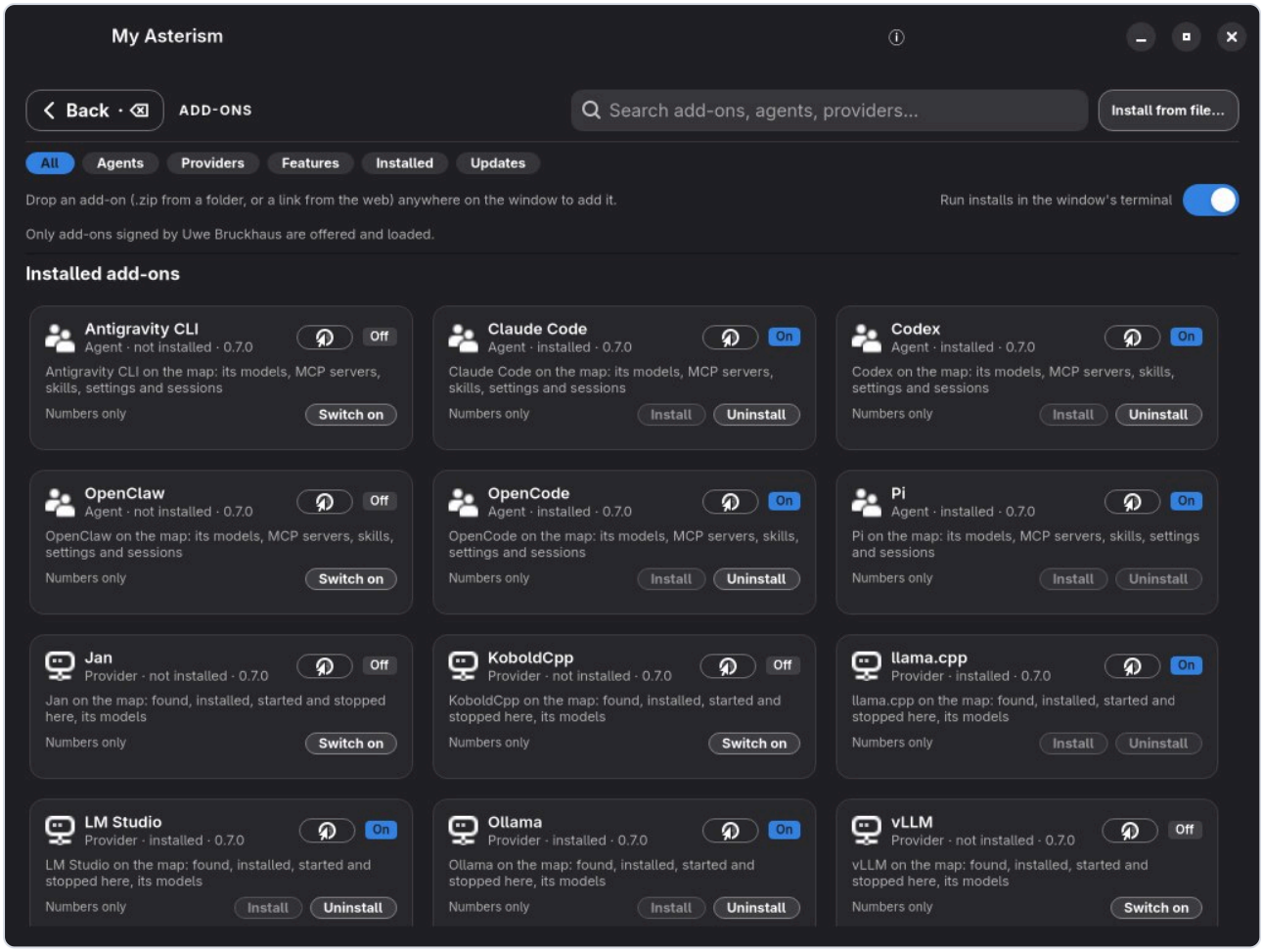



**Figura 17** · Complementos instalados: ativados ou desativados, com Instalar e Desinstalar para o programa.

O cartão de um agente ou LLM local ativado tem **Instalar** e **Desinstalar** para o próprio programa (o que não se aplica fica a cinzento), e cada cartão com página inicial tem um botão com um globo que a abre no seu navegador.

A pesquisa encontra nomes, descrições e o que um complemento lê; os filtros por cima reduzem a lista a agentes, fornecedores, funções, aos instalados ou aos que têm atualização. Clique num cartão para ver os detalhes: o que lê, o que envia, o que pode fazer e o seu nível de privacidade.

## 13.3 Instalar, ativar e desativar, atualizar, remover

**Instalar** (ou **Obter complemento**) mostra primeiro o que o complemento lê, envia e pode fazer; só depois do seu OK é descarregado, verificado e ativado. **Desativar** e **Ativar** atuam de imediato, sem reiniciar: os seus agentes, fornecedores, secções e cartões aparecem ou desaparecem em todo o lado. **Atualizar** substitui-o pela versão mais recente do catálogo depois de verificar a nova; **Remover** move-o para o lixo. **Instalar a partir de ficheiro...** aceita um zip de complemento que tenha descarregado; aplicam-se as mesmas verificações.

**SÓ DO AUTOR.** O Asterism só carrega complementos assinados por Uwe Bruckhaus (impressão digital da chave `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Antes de correr o que quer que seja de um complemento, a assinatura é verificada com o `ssh-keygen` do OpenSSH, cada ficheiro é comparado com a sua soma SHA-256 e nenhum ficheiro pode ter sido acrescentado. Uma transferência tem ainda de corresponder à soma SHA-256 do catálogo assinado.

## 13.4 Níveis de privacidade

| Nível | O que o complemento pode ler ou enviar |
|---|---|
| **Só números** | Contagens, tempos, tokens, linhas alteradas, custo — o que o Asterism mostra sem complementos. |
| **Títulos de sessão** | Os títulos que um agente deu ele próprio às suas sessões (nunca as conversas). |
| **Títulos e um resumo do Claude** | Esses títulos e números, enviados para o seu próprio Claude Code — só com um clique, após uma pré-visualização exata do que é enviado. |

Um complemento nunca pode fazer mais do que o seu nível permite. Adicionar um agente ou um fornecedor, ou instalar um programa, são permissões à parte que o seu cartão indica.

## 13.5 Tipos de complementos

| Tipo | O que traz |
|---|---|
| **Agente** | Um agente de programação: onde guarda as definições, sessões, servidores MCP e skills, como se instala e como inicia sessão. Tudo o resto o Asterism constrói a partir disto — o cartão em Configuração, a sua caixa no mapa, Uso e tokens, Projetos, Estado e Segurança. |
| **Fornecedor** | Um programa de LLM local ou os fornecedores online: como é encontrado e instalado e como os agentes o alcançam. O Asterism constrói o seu cartão, a sua caixa no mapa e a sua página de detalhes — incluindo **Uso e tokens**: o que os seus agentes enviaram através dele, a partir dos próprios números de sessão. |
| **Extensão** | Mais dados sobre um agente ou fornecedor que outro complemento traz, sem o substituir — por exemplo o seu plano e limites, ou um cartão na sua página de detalhes. Só preenche o que o complemento do agente deixa vazio; a página do complemento diz o que acrescenta e o que disso não é usado. |
| **Função** | Algo que abrange vários agentes, como o Insights (o cartão Tópicos). |

A secção de um complemento na página de detalhes de um agente ou LLM local vai para o cartão de que trata: Uso e tokens, Estado, Tópicos (só nos agentes) ou um cartão próprio com o seu nome.

Um complemento de agente descreve o seu agente como dados (um ficheiro chamado `agent.toml`); a maioria não traz código nenhum, e a página do complemento di-lo: *Não traz código* ou, para os poucos agentes que precisam de algo (por exemplo a sua própria base de dados de sessões), *Dados sobre o agente, e um pouco de código para o que os dados não conseguem dizer*.

## 13.6 Um agente novo no seu computador

Quando instala um agente que o Asterism ainda não conhece, ele procura no catálogo (Definições › **Procurar complementos online**, ligado por omissão: a lista é lida do GitHub de poucas em poucas horas e só o pedido é enviado). Se um complemento conhece esse agente, **Atualizações e sugestões** (o **!** do cabeçalho) di-lo (*«O Kilo Code está neste PC mas não no mapa»*), e aparece um cartão em **Sugeridos para este PC**; um clique instala-o. Se ainda não houver complemento, os detalhes do agente dizem-no, com uma ligação para pedir um.

## 13.7 Criar um complemento

Um novo LLM local ou um novo agente só precisa de um complemento: o Asterism constrói o seu cartão, a sua caixa e as suas linhas no mapa, os seus detalhes, a página Modelos e a página Segurança a partir do que o complemento diz sobre ele. Se souber programar um pouco, comece pelos modelos no repositório do Asterism (`templates/`, explicados em `docs/addons.md`): um ficheiro para um LLM local, um ficheiro de factos para um agente. Envie o complemento para `github.com/uBruckhaus/asterism-addons`; depois de revisto e assinado, aparece para todos em **Disponíveis**.

CAPÍTULO 14

# Privacidade e segurança

- **Sem permissões de administrador para a app.** Nunca corre como root nem vê uma palavra-passe. Instalar um agente que vem como pacote do sistema (como o Hermes ou o OpenClaw) executa o gestor de pacotes do sistema com `sudo` num terminal que vê, depois de lhe mostrar o comando.
- **Nenhuma credencial lida.** Lê as definições dos agentes, mas nunca os seus ficheiros de credenciais. Se um agente tem sessão iniciada percebe-se pela existência desse ficheiro. Os valores de chaves e cabeçalhos nunca são mostrados.
- **Nenhuma conversa lida.** Os números de uso são lidos apenas como números e tempos.
- **Rede só quando a pede** — pesquisar ou instalar modelos, servidores MCP ou skills, testar um servidor —, mais uma verificação de atualizações de poucas em poucas horas.
- **Transferências verificadas.** Os programas só são instalados se a transferência corresponder à soma publicada pela sua versão.
- **Pré-visualização e Desfazer.** Cada alteração às definições de outro programa é mostrada antes, copiada em segurança e pode ser desfeita. As definições que podem conter chaves são escritas legíveis só pelo utilizador.
- **Limitado.** Cada programa corre sem shell, e cada resposta é lida com limite de tamanho e de tempo.
- **Funções de IA só quando as escolhe.** As pesquisas do Smart Check correm com um clique, as verificações IA diárias só quando estão ligadas. O agente recebe um pequeno resumo (nomes, versões, modelos), nunca chaves nem valores de definições; as suas respostas são verificadas antes de serem mostradas e nunca são executadas. As tarefas correm num terminal que vê, e o agente pergunta antes de cada passo. Usa-as por sua conta e risco.

O README de cada repositório lista por completo o que é lido, iniciado e escrito.

CAPÍTULO 15

# Resolução de problemas

| Problema | O que fazer |
|---|---|
| **O mapa está apertado ou cortado.** | A janela tem menos de cerca de 1300 píxeis. Alargue a janela, ou reduza o zoom (`Ctrl`+`-`). |
| **Falta no mapa uma caixa que espero.** | O mapa só mostra o que está configurado: veja em **Configuração** (um complemento desativado, um servidor MCP que nenhum agente usa) e nos interruptores **Mostrar**. |
| **Iniciar aqui falta ou não faz nada.** | Instale o VTE para GTK 4 ( `vte4` ), ou use **Iniciar no terminal**. |
| **Uma linha está vermelha tracejada.** | Clique nela para ver sugestões, corrija o que dizem e depois **Testar de novo**. Ver Quando uma ligação falha. |
| **O LLM local não arranca com o agente.** | Verifique Definições › **Iniciar LLMs locais com os seus agentes** e se o agente está ligado no mapa. `systemctl --user status ai-agents-map-providers` mostra o vigilante. |
| **A luz colorida de um LLM local até ao seu modelo não aparece.** | Só aparece enquanto um programa que não está no mapa gera com esse LLM, e apaga-se cerca de dez segundos depois do seu último pedido. Verifique se o mapa está visível (uma janela larga), se o programa envia mesmo algo e se Definições › **Animações do mapa** está em **Agentes e outros programas**. |
| **O agente responde mal ou corta-se com um modelo local.** | O contexto pode ser pequeno demais. Abra o modelo em **Modelos… › Instalados** e use **Definir contexto** com o melhor valor para a sua placa. |
| **Uma transferência parou.** | Depois de uma quebra de ligação continua sozinha. Depois de reiniciar, use **Continuar** na faixa do topo. |
| **Quero desfazer uma alteração anterior.** | As cópias de segurança estão em `~/.local/state/ai-agents-map/backups/` . |

APÊNDICE

# Referência

## A.1 Atalhos de teclado

| | |
|---|---|
| `F1` | Ajuda |
| `Ctrl`+`,` | Definições |
| `F5` | Atualizar (o mapa e Configuração) |
| `Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0` | Ampliar o mapa, reduzi-lo, voltar a 100 % |
| `Ctrl`+roda | Zoom do mapa |
| `Shift`+clique | Acrescentar à seleção ou retirar dela |
| `Esc` | Limpar a seleção; voltar de uma página |
| `Retrocesso` | Voltar de uma página |

## A.2 Ficheiros e pastas

| Caminho | Conteúdo |
|---|---|
| `~/.config/asterism/` ASTERISM `~/.config/omarchy/ai-agents-map/` OMARCHY | Definições, a lista de servidores MCP, agentes pesquisados, ligações falhadas. |
| `~/.local/state/ai-agents-map/backups/` | Cópias de segurança de cada ficheiro que a app alterou. |
| `~/.local/state/ai-agents-map/jobs.json` | Transferências e instalações interrompidas. |
| `~/.local/state/ai-agents-map/reorganize.json` | O registo da última arrumação, para Desfazer. |
| `~/.cache/ai-agents-map/` | Resultados de pesquisa, listas de ferramentas, verificações de atualizações, o último Smart Check ( `smart-check.json` ) e as atualizações MCP das verificações IA ( `mcp-updates.json` ). |
| `~/.local/state/omarchy-plugin-ai/runs.json` | As pesquisas e tarefas de IA desta app e de outros plugins, com os seus tokens: as estatísticas do Smart Check. |
| `~/.config/systemd/user/ai-agents-map-providers.service` | O vigilante que inicia os LLMs locais com os seus agentes. |

## A.3 Glossário

| | |
|---|---|
| **Aprovação** | Um agente pergunta antes de alterar um ficheiro ou executar um comando (ver As palavras explicadas). |
| **Contexto** | Quantos tokens de texto um modelo consegue ter presentes de uma vez. |
| **GGUF** | O formato de ficheiro de modelos usado pelo llama.cpp, LM Studio, Ollama e KoboldCpp. |
| **Injeção de prompts** | Instruções escondidas numa página, num ficheiro ou na resposta de uma ferramenta que tentam conduzir um agente. |
| **Quantização** | Guardar um modelo com menos bits por peso (por exemplo *Q4_K_M*, cerca de 4,8 bits) para que fique mais pequeno, com pouca perda de qualidade. |
| **Mistura de especialistas** | Um modelo em que só uma parte dos parâmetros trabalha em cada token (*35B-A3B*: 35 mil milhões no total, 3 mil milhões ativos), o que o torna rápido para o seu tamanho. |
| **Sandbox** | Um espaço isolado onde correm os comandos de um agente, que só alcançam a pasta do projeto e o que é permitido. |
| **Token** | Um pedaço de texto, cerca de três quartos de uma palavra, que os modelos leem e escrevem. |
| **Serviço de utilizador** | Um programa em segundo plano que o systemd inicia só para o seu utilizador, sem permissões de administrador. |